

Machine Learning based Inverse Design of Tetra-petal Auxetic Structures

Khang Vo, Khanh Bui, Tomáš Remiš, Hung Le

Abstract

Metamaterials design through machine learning methods has been studied in multiple previous papers and reports; however, the most optimal machine learning architecture for usage in designing metamaterials through their properties remains to be accomplished. In our report, we aim to develop an inverse machine learning system which consists of a forward and an inverse model to design metamaterials through their properties, i.e., effective Poisson ratio, effective Young's modulus and volume fraction. The designs are parameterized by the usage of Non-Uniform Rational B-Splines (NURBS) to create tetra-petal auxetic structures. The pipeline includes a form of tetra-petal mesh generation, finite element simulations on the designs to determine the properties and dataset generation for machine learning training. We developed two inverse systems (inverse system A and B) which differ in both the model architecture and the training process. Both systems were evaluated by performing FEA simulations on the predicted designs and comparing their properties to the input. Furthermore, we evaluated the inverse systems by analyzing the risk of predicting invalid tetra-petal meshes. From our results, we found that the inverse system B had the best performance, as it produced much more accurate designs for volume fraction and effective Poisson's ratio, while having slightly worse metrics for effective Young's modulus. However, inverse system B also seems to be much more prone to generating invalid meshes, and thus manual inspection of models may be required to ensure their correctness. Lastly, we have created a GUI application which allows the user to both create tetra-petal designs through desired metamaterial properties through the inverse system B.

Table of Contents

Symbols and abbreviations.....	5
1. Introduction	6
2. Theoretical Background	7
2.1. Metamaterials - Types and Applications.....	7
2.2. Parametric Equations	10
3. Literature Review	15
3.1. Inverse Design	15
3.2. Other Approaches.....	18
3.3. Observations	21
4. Methodology.....	22
4.1. Parametric Design and Mesh Generation	22
4.2. FE Simulation	24
4.2.1. Setup.....	24
4.2.2. Simulation	25
4.2.3. Metamaterial properties	25
4.3. Dataset generation.....	26
4.3.1. Design parameter ranges	26
4.3.2. Parameter sampling.....	28
4.4. Machine Learning Framework	29
4.4.1. Overview of the Machine Learning Pipeline	29
4.4.2. Inverse Design System A	31
4.4.3. Inverse Design System B	36
4.4.4. Validation with FEA	40
5. Results	43
5.1. Properties predicted by the forward models	43
5.2. Performance of the inverse design systems	44
5.3. Comparing the amount of problematic designs.....	46
6. Discussion	49
6.1. Performance comparison between System A and B	49

6.2. Limitations.....	49
7. Extra deliverables	51
8. Conclusions and Outlook.....	54
9. Personal evaluation	56
10. References	57

Symbols and abbreviations

Symbols

E	Young's modulus
E^*	Effective Young's modulus
ν	Poisson's ratio
ν^*	Effective Poisson's ratio
V_f	Volume fraction
τ_{ii}	normal stress
R_i	NURBS basis function
C_i	i th control point

Operators

$\int_{\Gamma} d\Gamma$	integral along closed curve Γ
-------------------------	--------------------------------------

Abbreviations

ANN	artificial neural network
CNN	convolutional neural network
DL	deep learning
FFNN	feed forward neural network
FEA	finite element analysis
FEM	finite element method
GAN	generative adversarial network
LHS	Latin hypercube sampler
ML	machine learning
MSE	mean squared error
MAE	mean absolute error
NURBS	non-uniform rational b-splines
PGNN	physics-guided neural network
RVE	representative elementary volume

1. Introduction

Mechanical metamaterials are increasingly becoming prominent in multiple industries such as the aerospace and the automotive industries, due to their unique mechanical and electromagnetic properties deriving from their structure instead of their composition. Therefore, the main challenge of this paper concerns the design and manipulation of mechanical properties of metamaterials through ML-based inverse designing methods. According to Challapalli et al. [1], various research papers have demonstrated the advantages of applying ML techniques to predict the structural properties of materials. Combined with the extensive research other researchers have done on this topic, ML is a suitable tool for our task. Previous works have used ML models for designing materials via both forward and inverse design models, but there are some limitations. Zhang et al. [2] highlighted the prominent research gaps in machine learning models for materials design as being the robustness of ML models, interpretable design models and more advanced hybrid models. Our goal in this paper is to use machine learning to design an inverse ML model to generate metamaterial designs from predefined target properties, using the mesh generating method developed by Wang et al. [3] to produce tetra-petals unit cell designs through Non-Uniform Rational B-Splines (NURBS).

2. Theoretical Background

2.1. Metamaterials - Types and Applications

Metamaterials are materials whose properties are extracted from their structures instead of the material they are composed of. According to the paper from Kumar et al [4], they are a new class of man-made materials that can achieve physical and electromagnetic properties that cannot be achieved through natural means, such as cloaking and negative index of refraction. Combined with their design flexibility, these assets often contribute to the key characteristics of metamaterials, for instance enhanced bandwidth and control over electromagnetic direction. Because of these attributes, new devices and concepts have been developed using metamaterials and the applications of these unique properties. Some relevant sectors include medicine highlighted by Hosseinzadeh et al. [5], aerospace demonstrated by Palma et al. [6] and seen in **Figure 1**, and renewable energy, similar to what Ahmad et al. [7] wrote about and demonstrated in their image of theorized macroscopic metamaterial energy harvesters in **Figure 2**.

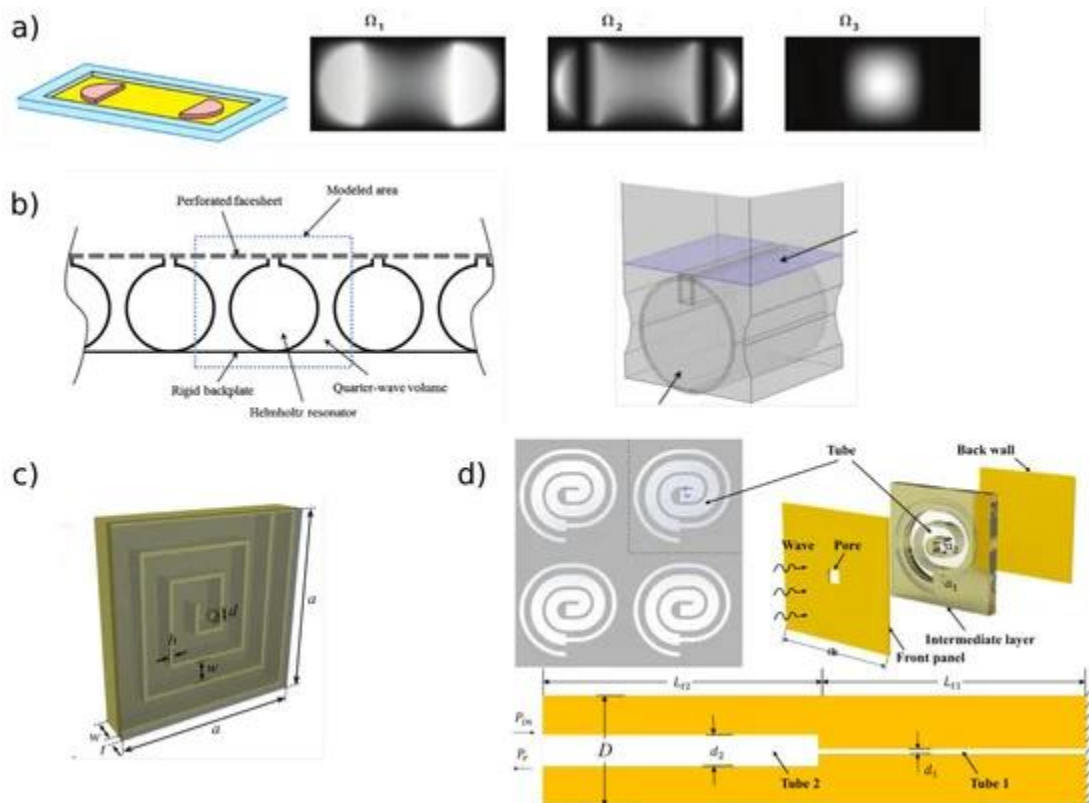


Figure 1: Various metamaterial designs for aeronautics [6]

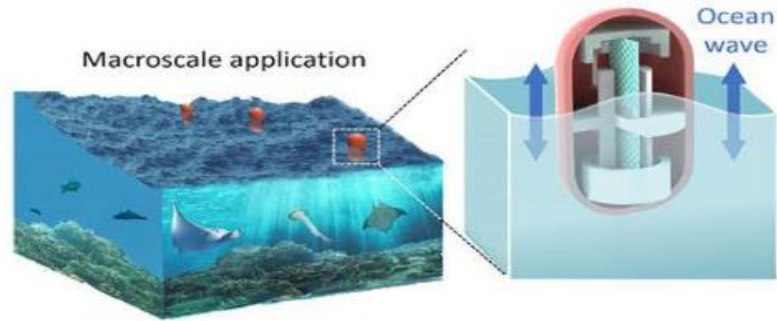


Figure 2: Model of theorized macroscopic metamaterial energy harvesters for renewable energy [7]

According to Kumar et al. [8], there are three main types of metamaterials: acoustic, electromagnetic and mechanical, each with their own unique properties and applications. The authors stated that acoustic metamaterials are usually developed via combining multiple materials with different mass densities and bulk moduli, leading to them having negative density, negative elastic modulus, and anisotropic mass. They state that these properties mean that they are widely used for seismic protection, cloaking, imaging, wireless power transfer, Anti-sonar, and acoustic metamaterials can be additionally divided into three more categories:

- (a) *Photonic crystals*, which contain resonating sites and regular arrays of acoustic scattering, leading to negative index behavior and periodicity in their structure.
- (b) *Resonant structures*, which utilize sound control and for acoustic filtering
- (c) *Space coiling*, which provides Fabry-Perot resonances and used in any condition in which waves can be guided without cut-off, comprising 3D and 2D acoustic waves and electromagnetic waves

Furthermore, they claim that electromagnetic metamaterials are synthetic materials developed by arranging homogenous metal structures. Their properties include magnetic permeability and electric permittivity, enabling them to sense humidity, pressure, temperature, wave guides, wireless data transmission and biological molecules in the environment, leading to usage in optics and utilities like microwaves.

Moreover, Kumar et al. [8] notes that mechanical metamaterials are by far the most widely studied areas due to their importance in real life uses. Properties such as being auxetic, stiffer, and increasing in strength at greater loads, and when combined with good sound and vibration absorption, these materials are used in defense and aerospace sectors. The image of 2 dimensional and 3 dimensional auxetic metamaterials are shown in **Figure 3**.

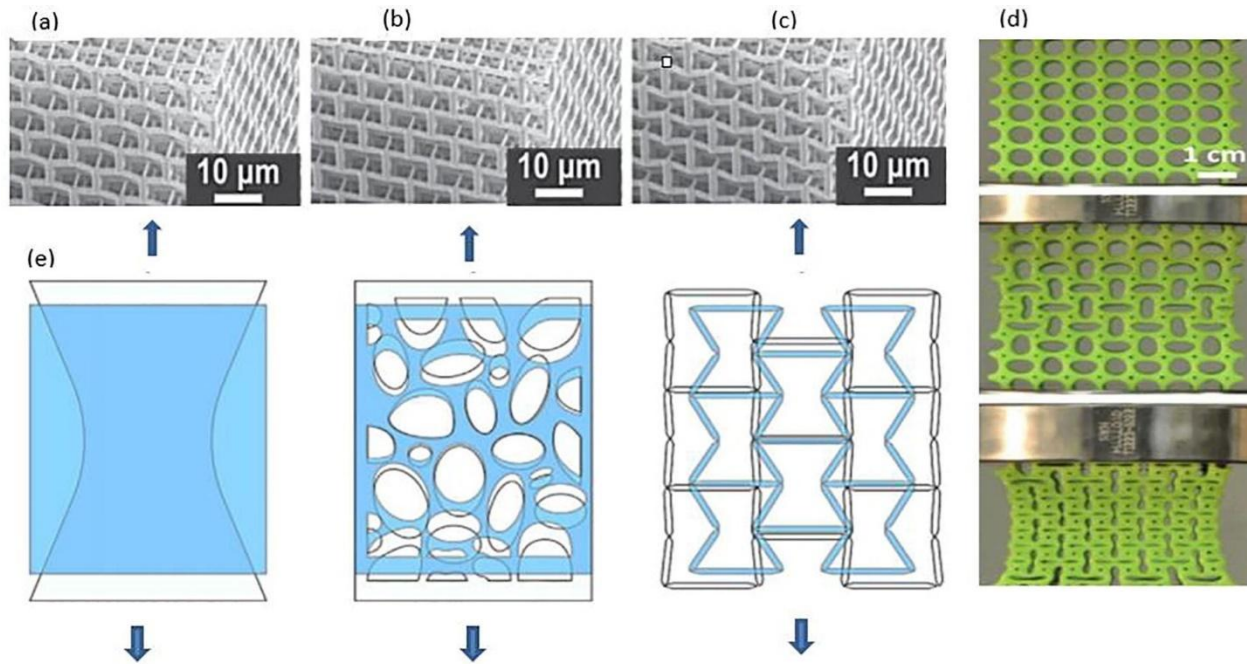


Figure 3: The schematic diagram of two dimensional and 3 dimensional auxetic metamaterials. [4]

Applications for metamaterials are extensive, spanning many sectors and industries. For the medical industries, many researchers have come up with methods to treat many ailments and diseases. Çalışkan et al. [9] documented the use of a slotted patch antenna for breast tumor treatment, and Xu et al. [10] applied helix antenna to monitor muscle activity and movement of hand fingers. In **Figure 4**, there is a more detailed illustration of the types of antennas that can be made from metamaterials. For the automotive industry, Liu et al. [11] demonstrated the need for new materials in the field for operating motors and electromagnetic compatibility means that metamaterials are more prominent. The practical applications in developing functioning parts from metamaterials in the automotive industry can be seen in **Figure 5**. Furthermore, they claim that mobile communication antennas, radar scanning, and electric motors were all enhanced due to the contribution of metamaterials. Moreover, the utilization of metamaterials is present in the aerospace sector, for instance, for noise reduction, invisibility shutter and a support structure reduction, albeit to a lesser extent due to many restrictions related to weight and performance.

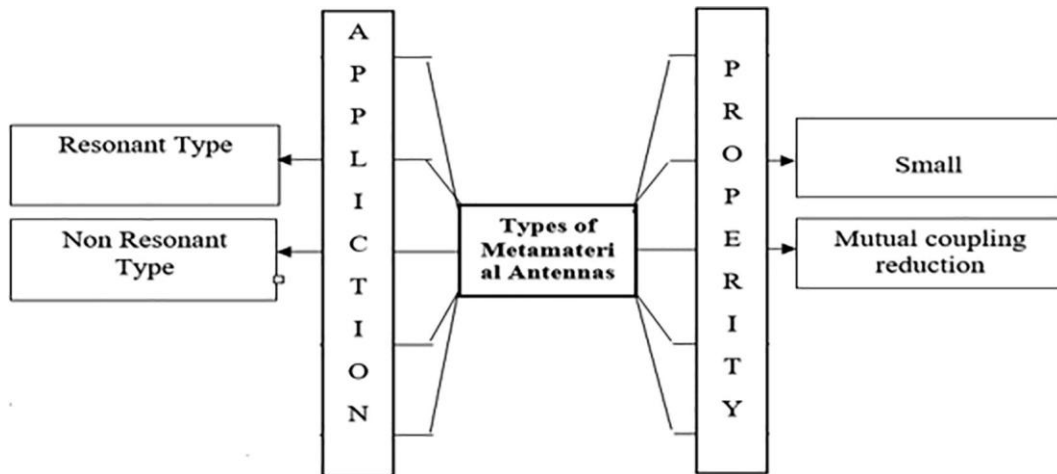


Figure 4: Types of antennas made from metamaterials [4]

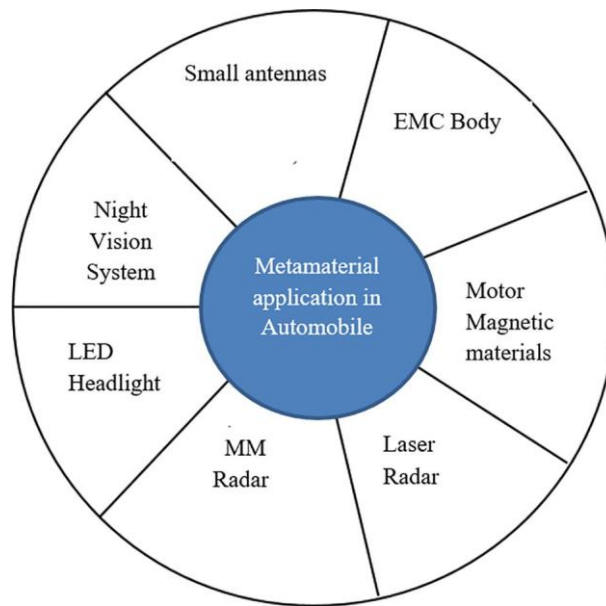


Figure 5: Metamaterial applications in Automobile field. [4]

2.2. Parametric Equations

As displayed above, the interest from modern technological fields in metamaterials is rapidly growing, leading to more research being done on methods to generate and parameterize equations and geometry to affect the properties of different metamaterials.

Zou et al. [12] documented various geometries and methods to parametrize geometry to affect specifically band gap properties of pentamode metamaterials, which is widely used

in 3D printing technology and detection technology using 2D pentamode metamaterials. Some techniques and methods included are band structures calculations, change-of-variables, transformation, and layer-by-layer rod structure characteristics, and we can see the exact microstructure and unit cell of pentamode metamaterials in **Figure 6**. These techniques can help the exploration of the relationship between the mechanical and electromagnetic properties of metamaterials and their performance, which is highly useful in various fields such as acoustic wavefronts and underwater acoustic carpet cloaks.

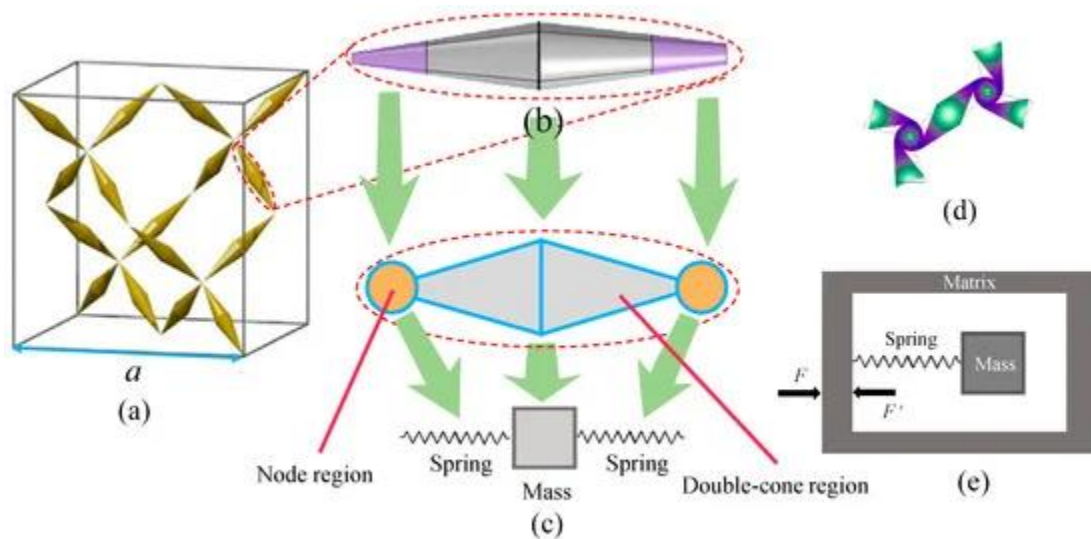


Figure 6: The microstructure and unit cell of pentamode metamaterials [12]

Furthermore, Széles et al. [13] wrote about how parametric equations can be used to generate fractal-inspired metamaterials with square unit cells, using rod elements and geometrical offset alongside generating new metamaterial structures. Techniques and methods used in this paper involve changing the number of elements and stiffening rod elements, which are consequently used to impart several advantageous properties, such as, better energy absorption through adjusting parameters, for example, Poisson's ratio. This can lead to the creation of a more advanced, tough and durable thermoplastic that is extremely versatile. We can see the rendered images of metamaterials designed by using these techniques in **Figure 7** for both ordered and offset geometries.

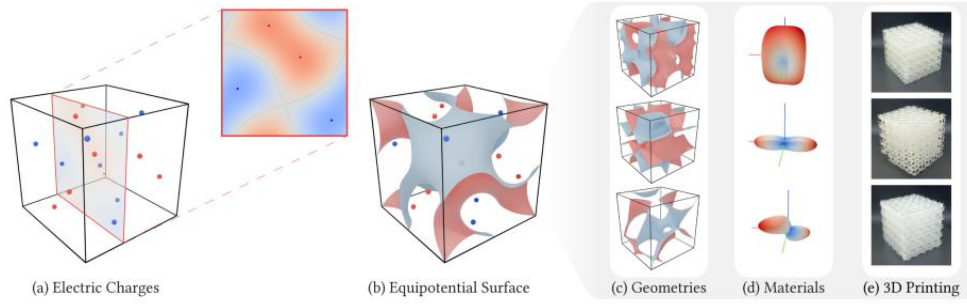


Figure 8: Design of shellular metamaterials through implicit functions parametrized by electric charges [15]

Yago et al. [16] wrote about machine learning in designing acoustic metamaterials, and mentioned how their strategy of optimizing Multiresonant Layered Acoustic Metamaterials (MLAM) revolved around the parameterization of each individual layers characterized by a specific number of unique parameters, and we can see the scheme outlined in **Figure 9**. Consequently, these layers will be used alongside an evolutionary algorithm for topology optimization. This parameterization scheme involved two distinct groups based on their topology: skin layers and resonating layers. These groups are then fed forward through a neural network to estimate effective properties required for the design of specific acoustic metamaterials.

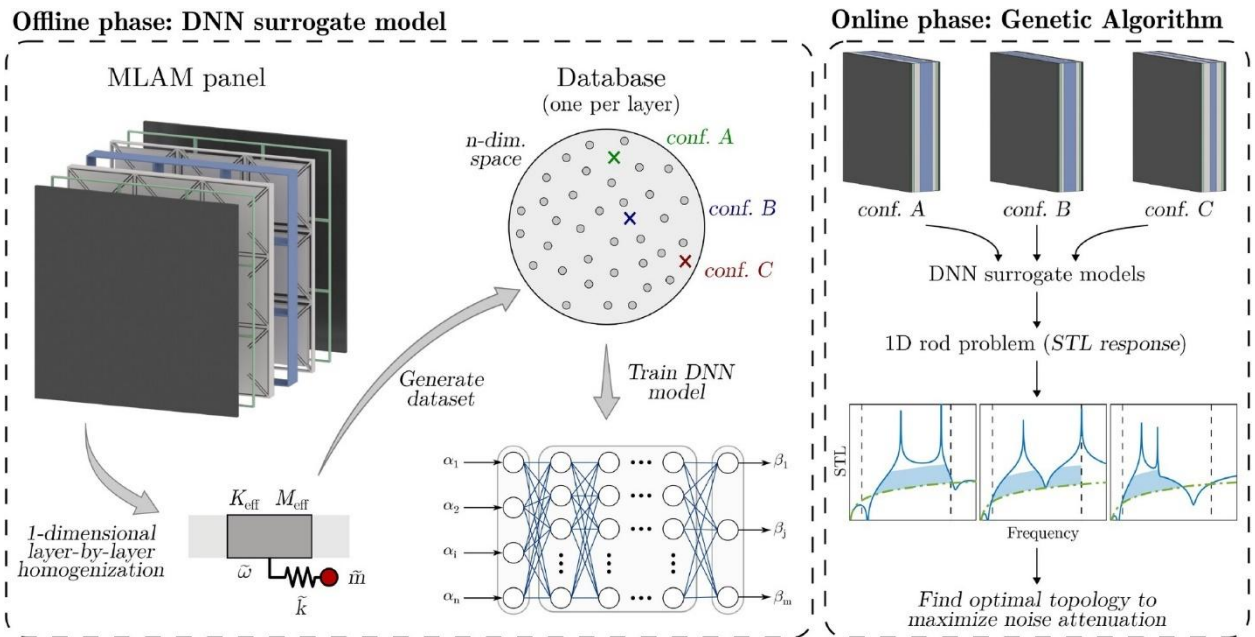


Figure 9: Scheme of the proposed strategy enhanced with machine learning techniques for the optimal design of MLAM panels [16]

Wang et al. [17] used the parameterization scheme NURBS for designing auxetic structures to achieve enhanced properties. Combined with the adopted optimization framework, design limit curves for Poisson's ratio for effective stiffness are obtained. Various structures with tunable effective properties have been designed through this method, like the design of a fiber reinforced composite and a non-uniform curved grid-stiffened composite. These structures have been gaining more attention due to their application in industries such as the aerospace industry and the bio-mechanical industry.

3. Literature Review

As part of the project, we performed a literature review on recent research that utilized ML in the metamaterial design process. There are two common approaches, namely inverse and forward design. Inverse design refers to identifying or generating designs that possess predefined target properties, which is suitable for optimization or discovery of materials [18]. On the other hand, forward design involves predicting behaviors of existing or undiscovered materials. When applied in a high-throughput manner, this design paradigm is useful for generating extensive databases that reveal the variation of properties across a wide range of compositions and structures [18]. ML models can then be trained on such databases to create fast surrogate models to be used in place of computationally intensive simulations [1]. Regardless of design approach, ML accelerates the workflow given the predictive and computational power of artificial neural networks, namely deep neural networks in handling high-dimensional data [19]. In some applications, employing ML in metamaterial design is the magnitudes greater computational efficiency compared to numerical methods [19], [20].

3.1. Inverse Design

Ha et al. [20] present a novel approach to inverse design of metamaterials dubbed “rapid inverse design”. This approach combines generative ML and small-scale 3D printing to generate an extensive range of target uniaxial compressive stress-strain curves while accounting for manufacturing-induced deviations. The pipeline comprises of an inverse prediction module and a forward validation module. The input to the inverse model includes features of the uniaxial compressive stress-strain target curves, fabrication parameters and minimum printable feature size of the specified 3D printer. On the other hand, the output is a set of optimal design parameters used for fabricating the 3D test sample of size $20 \times 20 \times 20 \text{ mm}^3$ consisting of eight unit-cells in total. The test sample is expected to display a mechanical response, closely resembling the input curves. The results of this study show > 90% average prediction accuracy between the experimental and target curves.

The approach proposed by Challapalli et al. [1] necessitates the use of Generative Adversarial Network (GAN) for discovery of optimal designs. Referring to **Figure 10**, the characteristic feature of GAN is the use of adversarial training data, created from the **Generator** component. The role of the **Discriminator (forward regression model)** is to distinguish between real and fake data from mixed batches of actual data and random noise. This framework facilitates the identification of realistic lattice unit cell designs with enhanced properties. Furthermore, the inverse model by the Challapalli et al. [1] takes compressive strength and mass as input. This is due to the study’s focus on lightweight metamaterials for loadbearing structures. The output is a binary-valued vector, whose size is the number of all possible elements formed from 27 vertices. The value of the

vector element is 1 when the RVE element exists in the structure, otherwise, the value is 0. The final form of the output provides an ML-friendly format for training. In the training

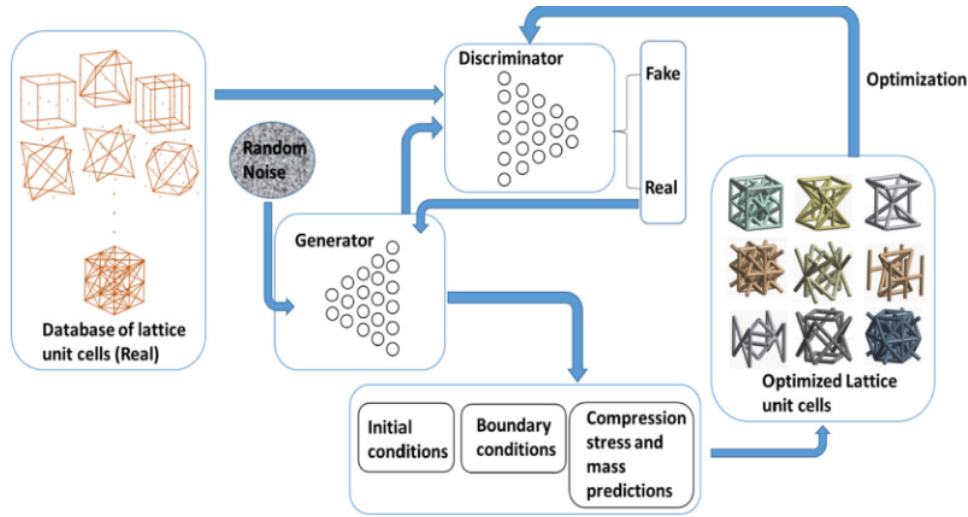


Figure 10: GAN framework for inverse design by Challapalli et al. [1]

Wang et al. [21] studied the application of ML in the design of shell-based mechanical metamaterial with specific loading curves. The dataset for the ML model is built from numerical simulations of these designs in Abaqus. For the ML, the authors employed FFNNs with a *tanh* activation layer. The input for the forward models is the configuration parameters mapped to a point in the design space, and the output is the compressive loading curve represented by 200 points. For the backward design, based on the FFNN output, the authors utilized genetic algorithms to iteratively generate metamaterial designs that closely match the desired loading curve. In the experimental results, the approach proved effective as the FFNN demonstrated excellent accuracy with a low prediction error of less than 5% in test cases.

Ma et al. [22] applied deep learning (DL) to construct chiral metamaterials using certain design parameters as a basis to achieve desired reflective properties. As illustrated in **Figure 11**, instead of a forward and inverse model, the authors used two bidirectional neural networks in a stacked ensemble configuration where each model makes their separate forward and inverse predictions. Moreover, the models use a combination of fully connected, tensor, and convolutional layers. For the forward segment, the output reflection spectra are combined based on the chiroptical response. For the inverse portion, the final output design parameters are calculated from the weighted sum of the models. The experimental results suggest that the addition of an additional auxiliary model improves the accuracy of predictions.

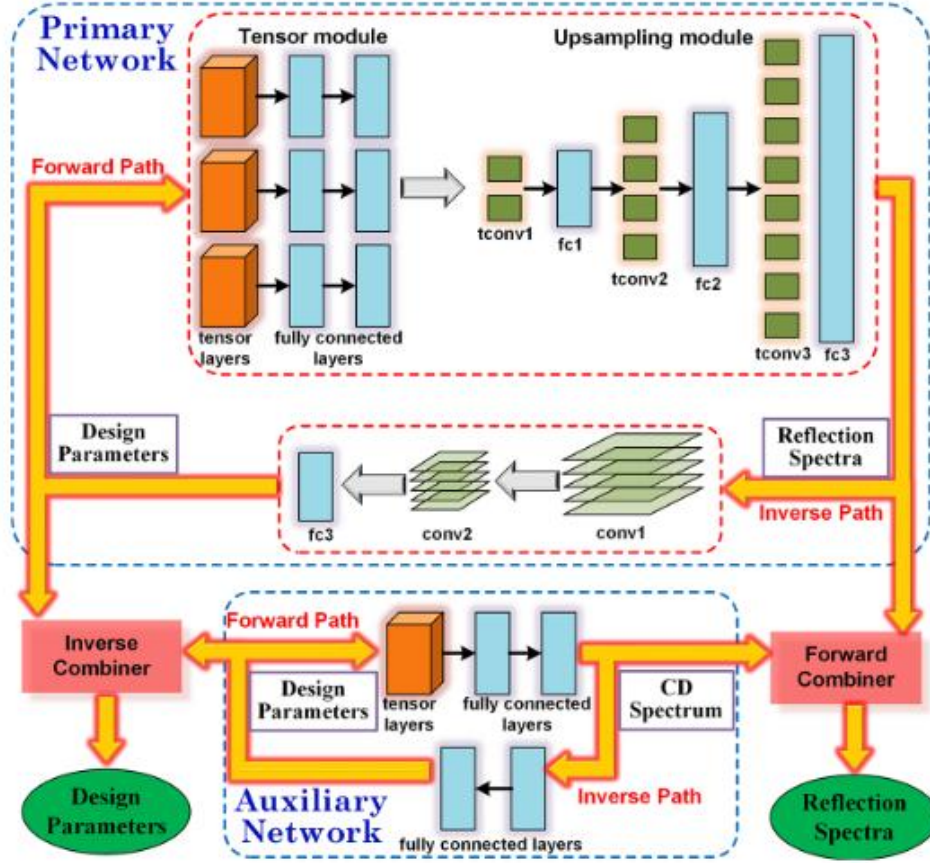


Figure 11. ML framework for chiral metamaterial design [22].

Abu-Mualla and Huang [23] proposed a PGNN-based ML framework to generate designs based on cubic unit cells with the appropriate elastic properties. In addition, to ensure the feasibility of the additive manufacturing of the design, the ML dataset generation incorporated geometric constraints. Another notable aspect of the framework is the utilization of PGNNs which, instead of relying only on data, integrated output from physical simulations into their training. The framework has an inverse design structure comprising of a forward and inverse model with the predicted properties of the generated design from the inverse model compared to the desired input properties.

Ashalley et al. [24] employed a somewhat different approach for the design and optimization of 3D chiral metamaterials with respect to the optical responses of the structure. The proposed end-to-end ML framework, illustrated in **Figure 12**, is comprised of two FFNNs. The dataset of 640 entries used in the study is generated from FEM. In the forward design segment, design parameters are given as inputs to predict the optical responses of the metamaterial. The innovation in this approach is the structure of the model which includes shared layers, task-specific (auxiliary and main tasks) hidden layers,

and output layer for each target output values. Moreover, the loss and optimizer of the outputs are calculated jointly. The approach achieved good results with an MSE score of 0.000441. As for the inverse design FFNN, the desired optical responses are given to predict the suitable design parameters. The model has a similar structure design method with task-specific layers. Then, these parameters are given as input to the forward model to verify that the predicted properties would match the desired values. Overall, the workflow of the framework is like our approach.

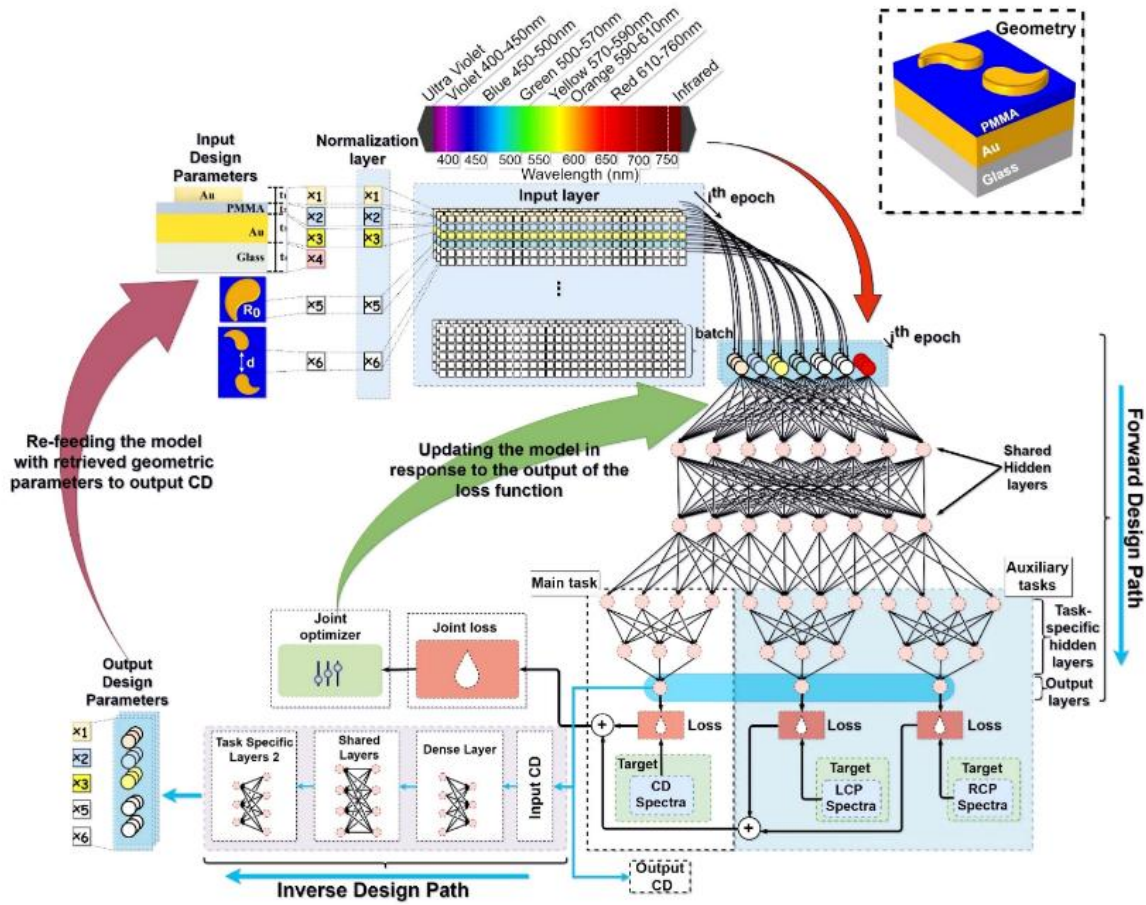


Figure 12. Multitask DL model for 3D chiral metamaterial design [24] .

3.2. Other Approaches

There are many examples of metamaterial design methods that employ ML but do not employ the inverse design approach. The study by Pahlavani et al. [25] focuses on predicting properties of metamaterials to ultimately uncover rare designs with optimal elastic properties. The training set was generated using re-entrant, cubic, and honeycomb

unit cells with cell angles of 60°, 90° and 120° and random proportions of hard and soft phases. The study also considers multi-tiled designs, considering different combinations of lattices and their properties. Based on the designs, the researchers employed FEA to acquire corresponding properties of the generated designs. Based on the predicted ranges of properties of randomly generated designs, the authors were able to identify patterns which correlate with desired properties.

Bessa et al. [26] utilized probabilistic Bayesian ML to design super compressible metamaterials. As illustrated in **Figure 13**, the design process metamaterial unit cell is data driven. By sampling from the design space of design variables in accord with a Sobol sequence, the authors performed exploratory computational investigation using finite element analysis (FEA) in Abaqus and built a response database of critical buckling stress and elastic energy absorption of the designs. Next, the database was used to train ML models to perform classification and regression. The classification model identified relevant regions of the design space, in which the regression model can focus on the prediction of response values. The model achieved good results with an average accuracy of 97.7% in classification. Based on the classification and regression results from the ML model, the authors were able to manufacture and test different designs.

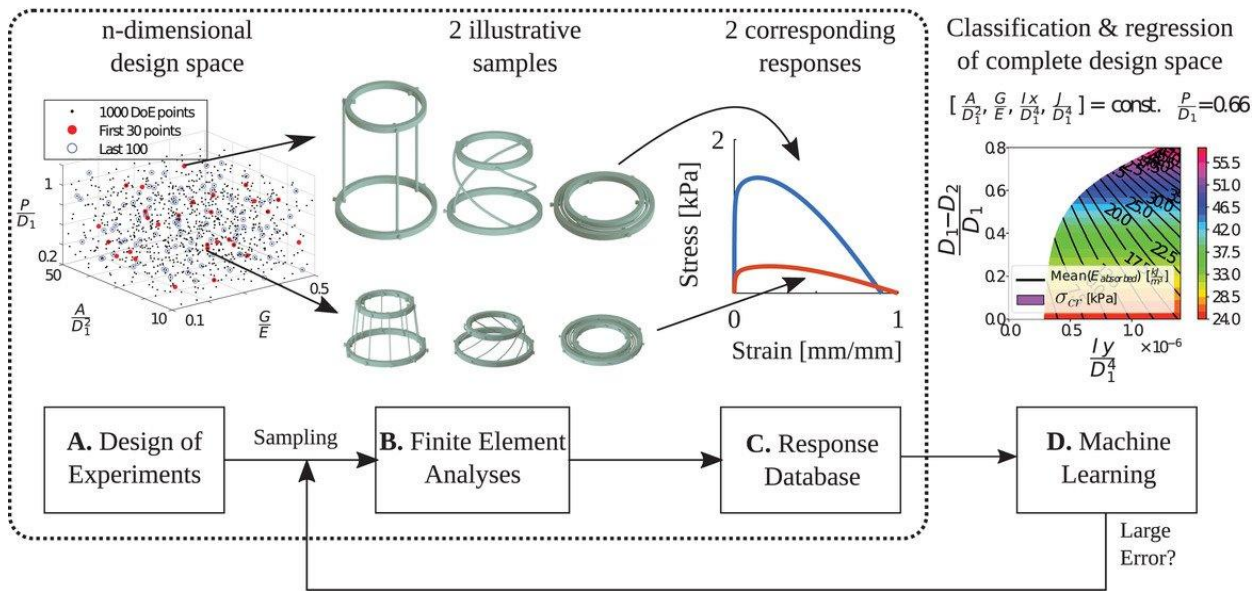


Figure 13. ML-guided super compressible metamaterial design workflow [26].

Wilt et al. [27] employed ML to design auxetic materials. First, Mathematica is employed to generate random asymmetrical Rorschach images which can be used as a basis for creating a mesh of the unit cell. Using FEA, the MAE is calculated based on the difference

in x-axis displacement in uniaxial experiments between optimal existing designs and the generated meshes. With the MAE and generated images as the dataset, CNN regression model, using TensorFlow and Keras, can predict the MAE. Though the model achieved excellent results (R^2 value of 0.91), the authors noted that at a higher number of random samples, the model produced greater errors due to the randomness of the dataset. Consequently, the authors concluded that future works should mathematically generate training images instead of employing a randomized approach to produce better engineering design.

Garland et al. [28] aimed to combine a CNN and a genetic algorithm to create metamaterial designs that optimize elastic stiffness during static loading and wave speed during impact. Since these objectives were conflicting, researchers decided to apply ML-guided designs to the problem to reach Pareto optimality [29]. Initially, they generated randomly 1000 designs in the form of 2D unit cells, which were bound by certain design constraints such as only one contiguous body. The relevant properties were then measured in Sierra FEA. The designs and FEA were used in the initial dataset for their CNN model as 3x3 unit cell tiling. The trained CNN was then utilized as part of the evaluation function along with the design constraints in a genetic algorithm. The general workflow is illustrated **Figure 14**. In addition, the final output designs were used as additional datapoints to augment the capability of the CNN model. The authors stated that although the size of the dataset was relatively small, it was sufficient for the training of the model. Moreover, the application of the CNN imparted noticeable computational efficiency compared to FEA simulations (more than eight times faster). Though our project does not use a genetic algorithm approach, general approach of the paper to database generation is like ours.

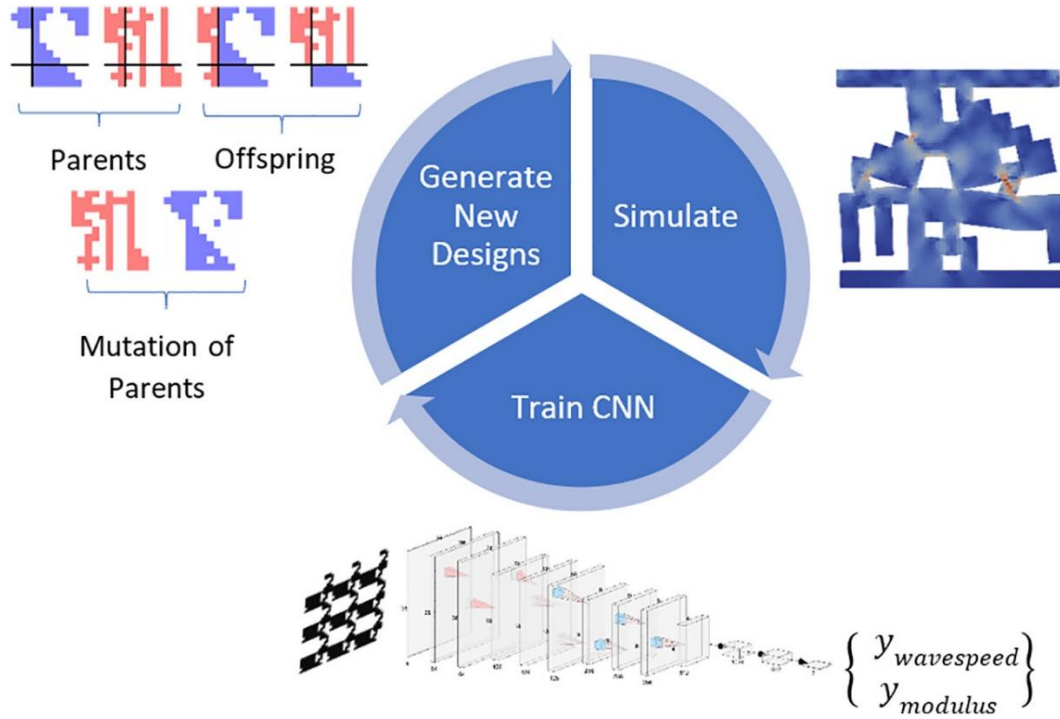


Figure 14: General workflow of the ML-driven approach by Garland et al. [28]

3.3. Observations

For our literature review, the team examined the existing literature with a focus on the methodology being employed such as data generation, ML architecture, and workflow. Overall, the methods and approaches to ML-based metamaterial design found in the literature are varied. Many papers utilized FFNNs as their models [20], [21], [23], [24]. On the other hand, alternative ML architectures were also implemented such as CNNs [27], [28] and GANs [1]. On the methods of data generation, a common approach found several papers, such as [21], [23], [24], [27], [28], opted to generate random datapoints with some parametric restrictions and calculated the generated design properties with FEA. These observations influenced our decisions on the eventual applied methods in the project.

4. Methodology

4.1. Parametric Design and Mesh Generation

To perform simulations and subsequent ML training, the design of unit cell needs to be determined. We have opted for the approach proposed by Wang and Poh [17] which implements tri- and hexa-petals structure through NURBS and adopted it for tetra-petals. The petal design has been chosen, as they are described only using 8 parameters ($w_1, w_2, w_3, w_4, h_4, d_1, d_2, d_3$), which simplifies training complexity. Furthermore, petal structures can yield desirable mechanical properties, such as auxeticity [30], which is more pronounced in tetra-petal designs than in tri- or hexa-petal ones [3]. Additionally, tetra-petals also exhibit isotropy due to their 4-fold symmetry. Moreover, due to the iso All generated designs are bound in a square with an area of 400 mm^2 . Two examples of generated tetra-petal designs are presented in **Figure 15**.

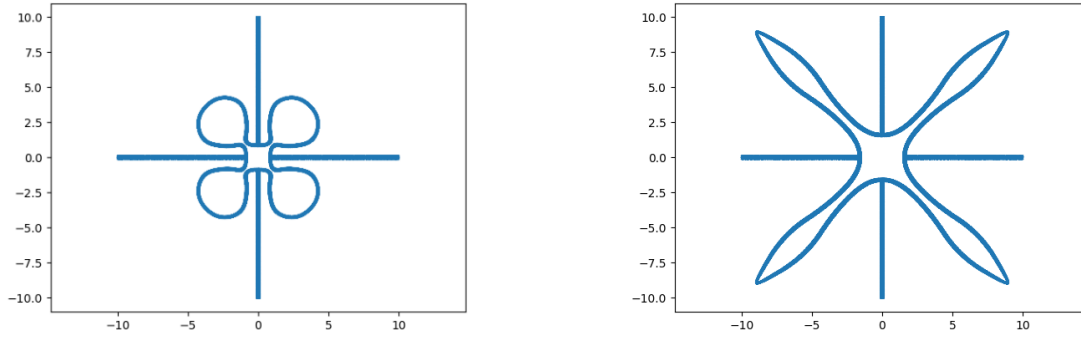


Figure 15: Examples of generated tetra-petal unit cells

The interior curve of the petal is described by ten control points $C_1, C_2, \dots, C_{10}$. The coordinates of the first 5 points are obtained as follows:

$$h_1 = |\tan(\theta)|, h_2 = \frac{h_4 - h_1}{3} + h_1, h_3 = 2 \cdot \frac{h_4 - h_1}{3} + h_1$$

$$C_1 = [w_1, h_1], C_2 = C_1 + n_1 \cdot d_1, C_3 = [w_2, h_2]$$

$$C_4 = [w_3, h_3], C_5 = [w_4, h_4],$$

where n_1 is the normal vector formed between C_1 and origin, and $\theta = \frac{\pi}{4}$ for tetra-petal. After defining the first five points, C_1 is projected onto the line defined by the angle θ .

Finally, the remaining five control points are obtained by mirroring the existing control points over the y-axis. After defining the control points, the NURBS interior curve is created using the knot vector $[0, 0, 0 : 0.125 : 1, 1, 1]$.

To define a valid surface, the exterior curve of the mesh also needs to be found. The exterior control point E_i is found through the following equation:

$$E_i = C_i + d_2 e_i,$$

where d_2 is the input parameter for thickness and

$$e_i = \frac{\int_{\Gamma_c} R_i \mathbf{n} d\Gamma}{\int_{\Gamma_c} R_i d\Gamma},$$

where Γ_c represents the interior boundary curve, $\mathbf{n}$ is the normal vector of the boundary curve and R_i is the NURBS basis function for the i^{th} control point. The exterior curve uses the same knot vector.

Afterwards, every point needs to be offset by d_4 on the y-axis, where

$$d_4 = \frac{d_3}{2 * |\cos(\theta)|},$$

where d_3 is the thickness of the connecting bar.

Furthermore, the parent petal mesh is parallel to the y-axis, however, it needs to be rotated by θ so the connecting bars are perpendicular to the bounding box. As the last step of parent petal generation, the generated mesh needs to be connected to a connecting bar. Each bar has the length of $l_b = \frac{\sqrt{400}}{2} - w_n$, where w_n is the x coordinate of the last curve point. The bar is attached to the last point of the interior curve. Lastly, the remaining petals are obtained by rotating the parent petal by θ three times. The dimensions of a single parent petal are highlighted in **Figure 16**. After the closed interior and exterior curved are found, they are used to generate the full mesh through triangulation.

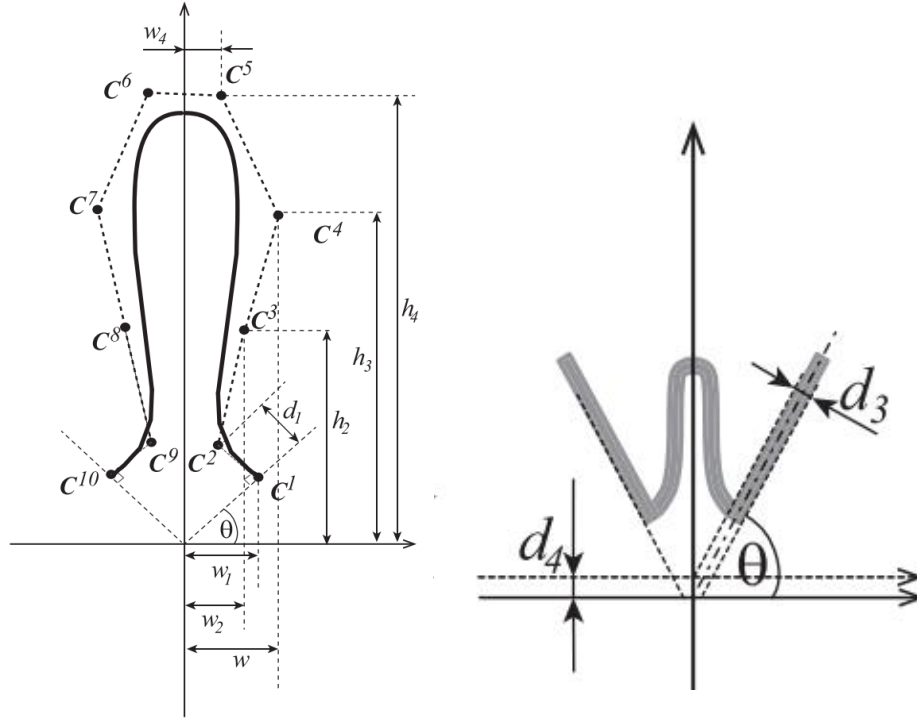


Figure 16: Left - coordinates of interior control points; Right - offset applied to parent petal and thickness of connecting bars [17]

4.2. FE Simulation

The following section describes the process of running simulations and extracting effective properties on the generated tetra-petal designs.

4.2.1. Setup

After mesh design is generated, it needs to be evaluated in a finite element analysis software to determine the metamaterial properties. For this purpose, we have decided to use the Abaqus FEA software. The meshes can be easily imported into Abaqus by writing the following into a .inp file:

- The coordinates of all mesh vertices
- The vertex indices forming mesh triangles
- Young's modulus and Poisson's ratio of the material

The chosen material properties are $E = 3 \text{ GPa}$ and $\nu = 0.35$, which corresponds to PLA plastic.

4.2.2. Simulation

Once a mesh is imported into the software, a uniaxial tension simulation is performed. Due to the isotropic nature of the design, the simulations yield identical results both along the x and y axes. Thus, only a single simulation along the x-axis is necessary. Two separate boundary conditions are applied to the borders of the connecting bars:

- The horizontal bars are displaced by 5 mm
- The displacement of vertical bars is set to 0

An example of this simulation can be seen in **Figure 17**. After the simulations, the field outputs for stress (S), displacement (U), and integration point volume (IVOL) are saved.

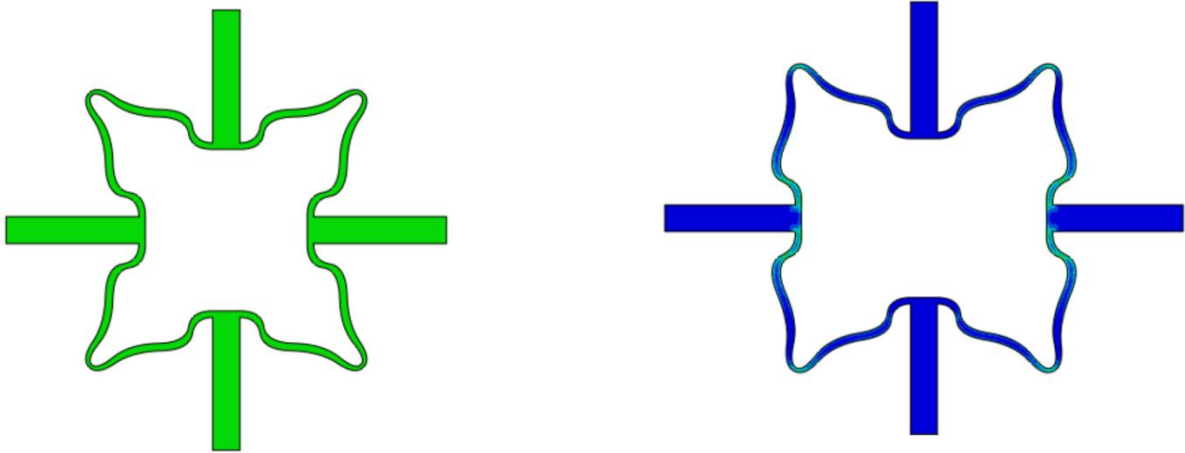


Figure 17: Left – a generated tetra-petal imported in Abaqus; Right – resulting mesh after uniaxial tension along the x-axis

4.2.3. Metamaterial properties

Once the simulation finishes, the metamaterial properties can be calculated based on the field outputs. We chose to calculate the effective Young's modulus (E^*), effective Poisson's ratio (ν^*), and volume fraction (V_f).

The effective Young's modulus and Poisson's ratio can be computed using computational homogenization [31]

$$\nu^* = \frac{\tau_{22}}{\tau_{11}}$$

$$E^* = \frac{(\tau_{11} - \nu^* \cdot \tau_{22})}{\varepsilon},$$

where $\varepsilon = \frac{u}{h} = \frac{10}{20}$ is the applied strain and τ_{11}, τ_{22} are the average normal stresses along the x and y axis respectively. The average normal stresses are calculated as

$$\tau_i = \frac{1}{V} \int_V \tau_i dV,$$

where $V = h^2 = 400 \text{ mm}^2$ is the RVE volume. Lastly, the volume fraction is obtained by calculating

$$V_f = \frac{V_m}{V},$$

where V_m is the total volume of the tetra-petal design.

4.3. Dataset generation

The dataset is created by generating and performing simulations on 1000 different tetra-petal designs. Both the design parameters and the simulation results are concatenated into a single 11 element vector and written into a CSV file as a single row. An example of a row can be seen in **Table 1** which corresponds to the same design as in **Figure 16**.

Table 1: example of a single CSV sample

w1	w2	w3	w4	h4	d1	d2	d3	ν^*	E^*	V_f
2.16	2.33	0.55	0.35	7.77	0.98	0.39	1.23	-0.32	0.16	0.12

4.3.1. Design parameter ranges

In order to ensure that the generated meshes are both bound within the square RVE and proper, each design parameter needs to be restricted into a range where the mesh is well-behaved. Some examples of invalid mesh can be found in **Figure 18** with the corresponding design parameters in **Table 2**. The main issues with improper meshes are that they can lead to failure of the mesh triangulation (Petal 1), interior/exterior curves overlap which can lead to unexpected resulting meshes (Petal 2 and Petal 3), or the mesh can grow outside of the RVE, in which case the computed properties can be incorrect. Furthermore, the connecting bars of invalid meshes may not reach the RVE border, meaning the generated design cannot be tiled properly. Therefore, it is important to choose parameter ranges range that would not lead to such failures.

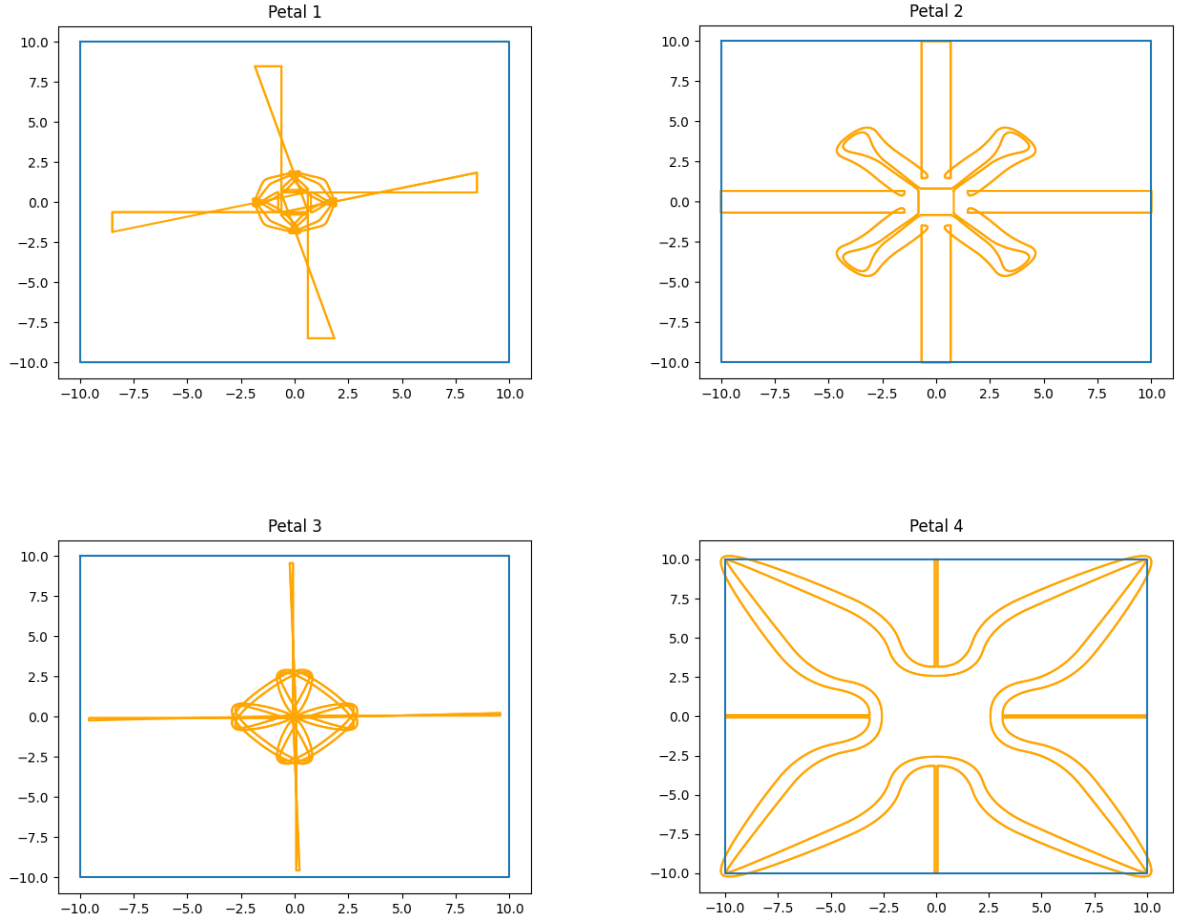


Figure 18: Examples of invalid tetra-petal designs, blue box represents RVE bounding box

Table 2: Design parameters corresponding to Figure 5

Petal	w1	w2	w3	w4	h4	d1	d2	d3
1	0.1	1.74	0.8	0.1	0.74	0.1	0.37	1.23
2	0.1	0.1	0.1	1.02	4.69	0.1	0.89	1.35
3	0.1	1.44	2.64	1.59	1.93	0.1	0.37	0.15
4	1.77	2.25	1.22	0.1	14	1.78	0.79	0.13

The design parameter ranges which we opted for are a combination of ranges presented by Wang et al. [17] and Zhang et al. [32] to create a more conservative range which does not generate invalid meshes. The ranges are as follows:

$w_1 \in [1, 2.8]$, $w_2 \in [0.5, 2.5]$, $w_3 \in [0.5, 3]$, $w_4 \in [0.25, 1.19]$, $h_4 \in [7, 10.5]$, $d_1 \in [0.3, 1]$, $d_2 \in [0.2, 0.85]$, $d_3 \in [0.2, 1.5]$.

4.3.2. Parameter sampling

To ensure that the dataset has good coverage of the parameter space, Latin hypercube sampling (LHS) is used to generate design parameters for all samples. LHS creates samples representative of the space by dividing it into n equal segments from which a single random sample is selected. The distribution of the properties of the generated samples can be seen in **Figure 19**.

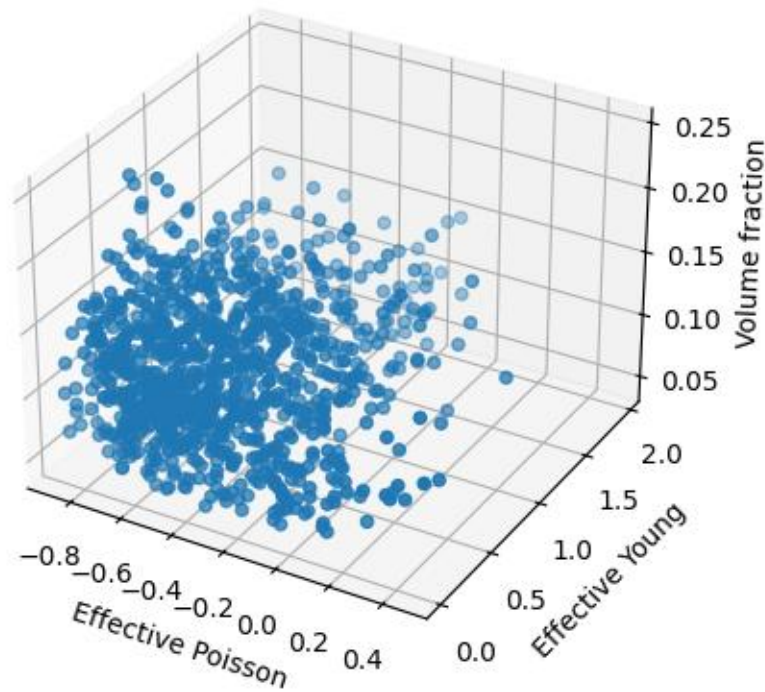


Figure 19: Distribution of dataset properties

Lastly, we split our design dataset into train/validation/test splits with the split ratios 0.7/0.1/0.2. Both the forward and inverse models are trained only on the train data split. The validation split is used together with the train split, and its predictions inform us whether the models are overfitting on the training data. After the training phase is complete, we calculate regression metrics on the test split which informs us about the

efficiency of the models on unseen data. The distribution of properties present in all three splits is shown in **Figure 20**.

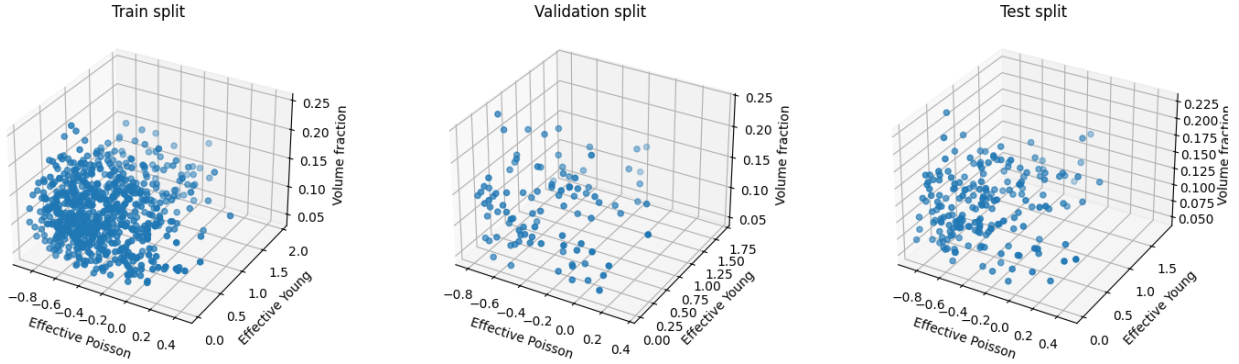


Figure 20: Distribution of properties along the train/validation/test splits

4.4. Machine Learning Framework

4.4.1. Overview of the Machine Learning Pipeline

With the generated dataset, the next step was the ML implementation. This section is an overview of our ML-based inverse design framework. By utilizing inverse design, the team aimed to implement an end-to-end ML solution that can predict the design parameters of a metamaterial with the desired properties. The flow of the ML system was partially based on the work of Kumar et al. [4]. Specifically, there were two parts to our implementation: the forward model and the inverse model. The forward model was trained using the dataset with the design parameters as the input and the material's properties as the output. Conversely, the inverse model would have the desired properties as the input and the predicted design parameters as the output.

Firstly, the forward model was trained on the generated dataset. Then, the inverse model would employ the trained forward model in its training process as an approximator that can predict the properties of the predicted designs created from the parameters of the inverse model. The difference between the target properties and the properties of the predicted designs would serve as the metric in which the inverse model would attempt to minimize during training. In addition, the application of the forward model in this instance would also circumvent the need for computationally expensive FEA analyses required for predicted design's properties validation. The general pipeline for the ML framework can be seen in **Figure 21**. The architectures and training strategies of the forward and inverse models will be illustrated in more detail in **Section 4.4.2** and **Section 4.4.3** respectively.

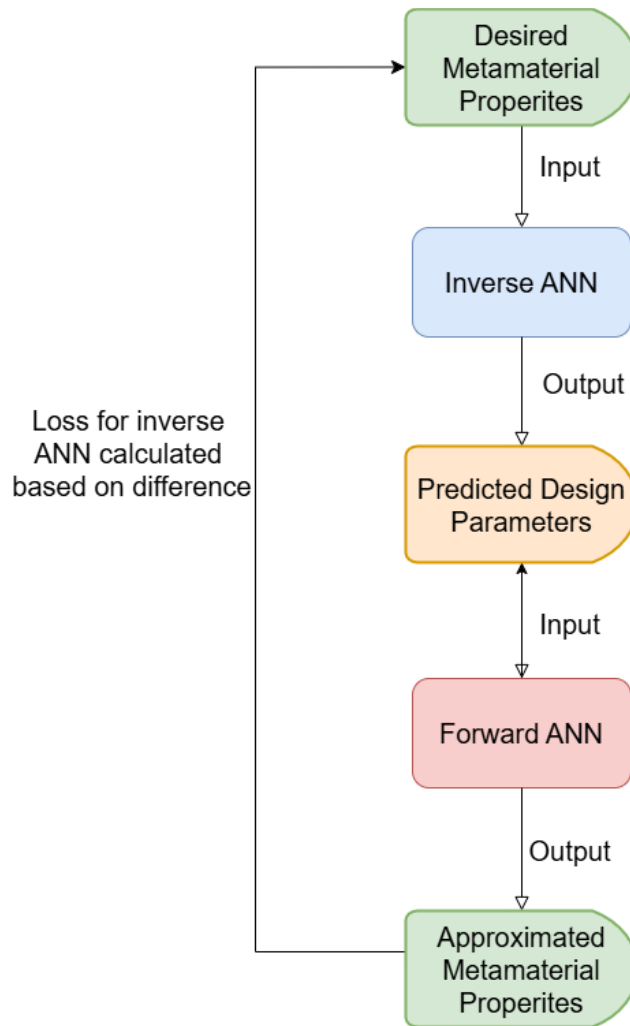


Figure 21: Machine Learning Pipeline

We also include an additional validation step with FEA via Abaqus simulations, whose setup will be further discussed in **Section 4.4.4**. This allows us to verify whether inverse model outputs truly exhibit mechanical behavior close to the desired properties and whether the forward model is a reliable approximator of properties.

As part of the literature review, we investigated what ML architectures are commonly applied in metamaterial problems. As seen from Section 2, the approaches adopted by researchers were varied. One of the most common architectures that we found were FFNNs [21], [32], [33], [34]. In addition to the approach being a proven method, FFNNs were chosen based on the relative ease of implementation compared to GANs or RL, and familiarity of the team members with the architecture. The ML component of the project was executed using the PyTorch library in Python.

The architectural decisions were based on common machine learning practices and empirical testing results. NVIDIA's deep learning documentation suggests that the hidden layer dimensions and batch sizes were chosen to be powers of two, to ensure efficient training of deep learning models on common hardware, especially GPUs [35]. However, they also provided a disclaimer that choosing powers of two that are greater than 512 does not introduce additional computing benefits. Therefore, we cap the value of hidden layer dimensions at 512 as it provides sufficient complexity to our model.

In the sections 4.4.2 and 4.4.3, we will present two inverse design systems we developed iteratively during this project, named System A and System B. System B is believed to be an enhanced version of System A after studying in great detail caveats of System A.

4.4.2. Inverse Design System A

The architecture of the forward model of system A, abbreviated as Forward A, is illustrated in **Figure 22**. It consists of:

- Input layer: 8 neurons (design parameters: $w_1, w_2, w_3, w_4, h_4, d_1, d_2, d_3$)
- Hidden layer: 128 neurons linear transformation layer with batch normalization, ReLU activation function and dropout.
- Hidden layer: 64 neurons linear transformation layer with batch normalization, ReLU activation function and dropout.
- Hidden layer: 32 neurons linear transformation layer with ReLU activation function.
- Output layer: 3 neurons (predicted properties: Poisson's ratio, Young's modulus, volume).

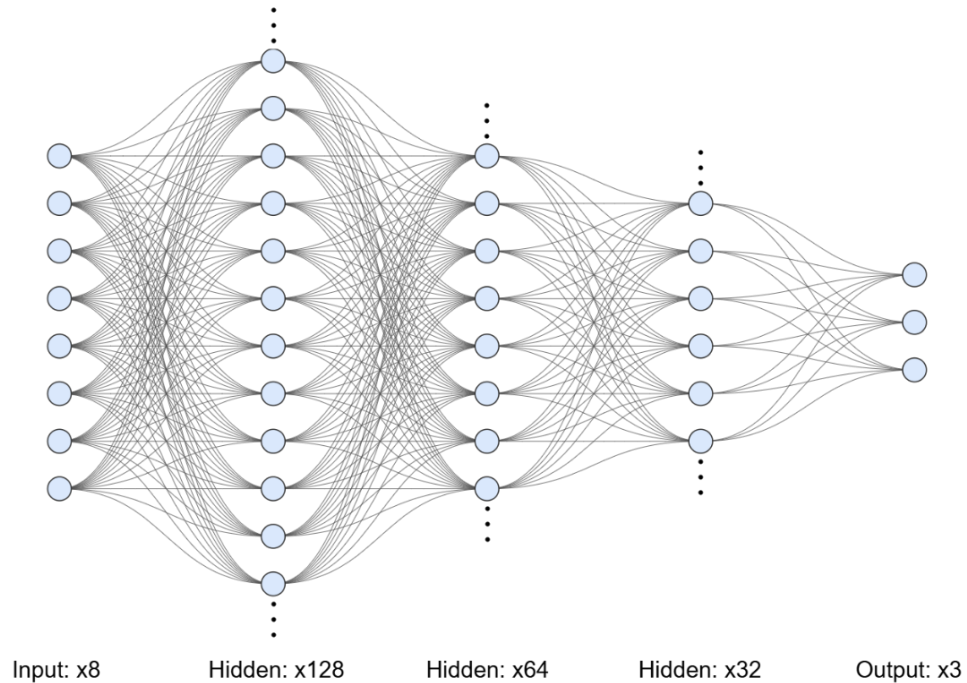


Figure 22: Forward A FFNN Architecture

The predictions of Forward A for Poisson and Young's modulus are already strong, with R2 values exceeding 0.96 as we can see in **Table 3**. However, the prediction for volume lags substantially at only approximately 0.537.

Table 3: Performance metrics on the test set on forward A without tuning

Effective Poisson's ratio			Effective Young's modulus			Volume fraction		
MSE	MAE	R2	MSE	MAE	R2	MSE	MAE	R2
0.001	0.024	0.9807	0.003	0.037	0.976	0.001	0.002	0.537

In order to achieve better results, we decided to use Input Data Standardization as demonstrated by the paper by Anysz et al [36]. According to the paper, good preparatory work on the input data set is essential to achieving good results in the application of ANNs, and there are a variety of methods and formulas that can be used to standardize the input data, such as:

- Z-score standardization
- Vector standardization
- Manhattan standardization
- Maximum linear standardization
- Weitendorf's linear standardization
- Peldschus' nonlinear standardization
- Zavadskas and Turskis' logarithmic standardization

With our neural network model done mainly in Python using PyTorch, which has Z-score standardization built-in in the StandardScaler class, we used that to standardize our inputs. It is also extremely efficient, as the paper by Al-Faiz et al. [37] showed through the significant decrease in the number of epochs required for the network to learn after Z-score standardization.

We implemented this by using the StandardScaler for both the inputs and outputs, then using them for training/testing/validation splitting. This uses the statistics of the whole dataset to transform and reverse transform inputs and outputs. With this standardization method, we can plot the training and validation loss to see whether the method works. The training loops employs the Adam optimizer with learning rate $5e^{-4}$ and MSE loss as the loss criterion. The moderately high learning rate is expected to create a rapid drop in losses given after only a small number of epochs. The training and validation losses can be seen in **Figure 23**.

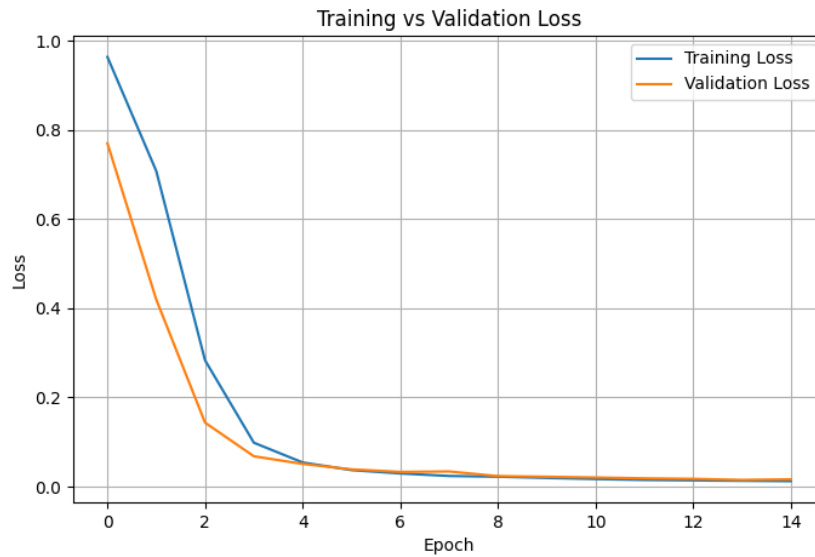


Figure 23: Training and validation loss using Z-score standardization during first iteration of Forward A training

As is evident from the losses, the training and validation losses decrease exponentially quickly, reaching 0 at around the 15th epoch. We notice at the end of the training process, the gap, there is no visible gap between the validation and training losses, indicating a well-generalized model. Specifically, this means it is neither overfitting nor underfitting. This means that we can also plot the true values compared to their predicted values for all three properties. Additionally, we evaluated the model on a test set, separated from the training and validation sets. **Figure 24** below shows the predicted and true values of the outputs. The x-coordinate and y-coordinate of each blue dot represent the true output value and the predicted output value respectively. The red line represents the function $y = x$. Any point directly on the line is a perfect prediction, where the predicted value is exactly equal to the true output. Therefore, the closer the blue points are to the line, the smaller the difference between predicted and true values is. Observing the graphs visually, blue dots are located quite closely to the line, suggesting high performance.

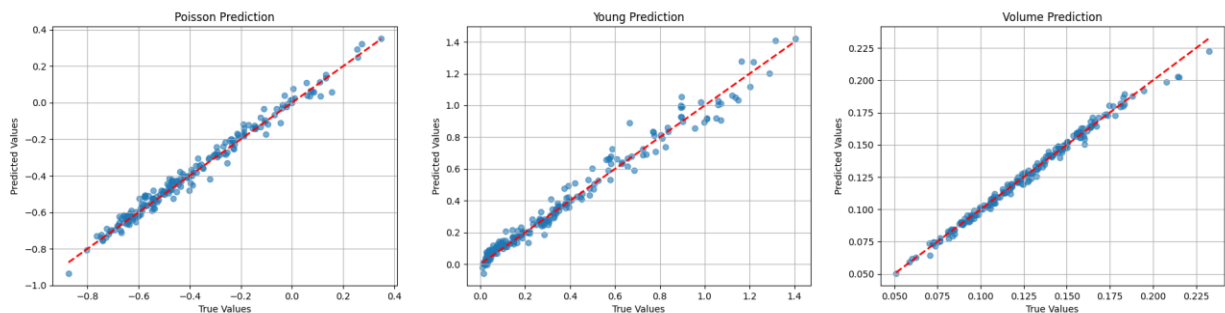


Figure 24: Forward A predicted vs. true values for test after Z-score standardization implementation

Table 4 below summarizes the performance metrics of this first version of Forward A. MSE and MAE losses on average are low across all three properties while R2 values are equal to or larger than 0.97. This indicates that the model generalizes well to unseen data. Specifically, the model is the best at predicting volume fraction with MSE close to 0 and R2-score at 0.985. The model struggles slightly more with effective Young’s modulus and Poisson’s ratio.

Table 4: Test performance metrics of Forward A

Effective Poisson’s ratio			Effective Young’s modulus			Volume fraction		
MSE	MAE	R2	MSE	MAE	R2	MSE	MAE	R2
0.002	0.031	0.976	0.003	0.036	0.970	0.000	0.003	0.985

The second component of the inverse system is the inverse model itself which is denoted as Inverse A. The architecture of Inverse A is almost identical to Forward A, with the exception of the input and output sizes. We experimented with this architecture initially due to the belief that slight changes in the input sizes do not detract from the suitability of the architecture.

In terms of the training loop of the first iteration, we chose Adam optimizer with a learning rate of $1e^{-3}$, which is lower than that of Forward A. During this iteration the loss function is a sum of MSE loss between forward predicted properties and requested properties and the loss between the dataset design parameters and inverse-predicted design parameters. This reflects the belief that many designs can satisfy one set of properties. In addition, we also do not want the design parameters to deviate too much from the expected ranges. We kept the loss between dataset design parameters and inverse-predicted design parameters as part of the loss function to regularize the outputs. The formulation of the loss is as follows:

$$loss = properties\ loss + penalty$$

$$loss = MSE(desired\ properties, forward\ predicted\ properties) \\ + MSE(inverse\ predicted\ design\ parameters, true\ dataset\ design\ parameters)$$

As seen from **Figure 25**, the architecture of the Inverse A consists of:

- Input layer: 3 neurons (predicted properties: Poisson's ratio, Young's modulus, volume).
- Hidden layer: 256 neurons linear transformation layer with ReLU activation function and dropout.
- Hidden layer: 64 neurons linear transformation layer with ReLU activation function.
- Hidden layer: 32 neurons linear transformation layer with ReLU activation function.
- Output layer: 8 neurons tanh layer (design parameters: $w_1, w_2, w_3, w_4, h_4, d_1, d_2, d_3$). The output layer consists of tanh functions, and the results are scaled.

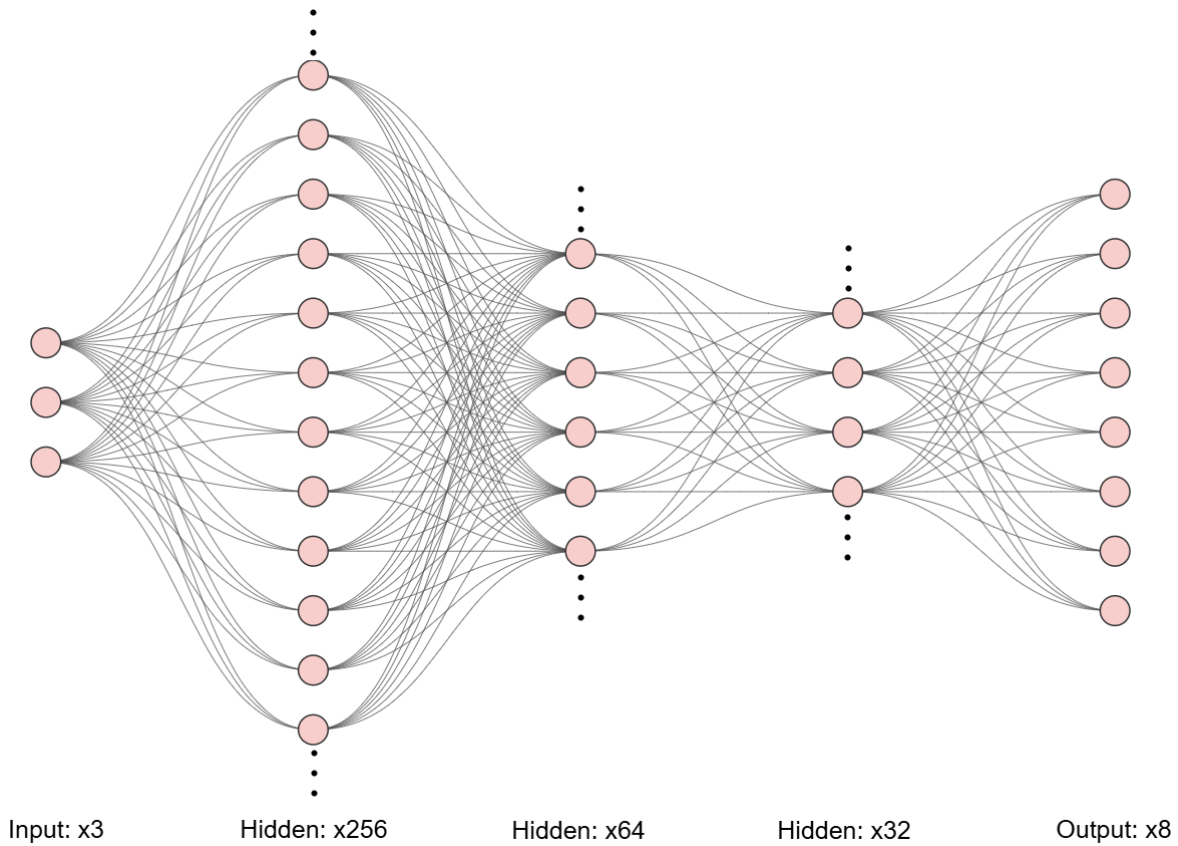


Figure 25: Inverse A architecture

The performance on the test set of Inverse A can be illustrated via **Table 5** below.

Table 5: Test performance metrics of Inverse A

Effective Poisson's ratio			Effective Young's modulus			Volume fraction		
MSE	MAE	R2	MSE	MAE	R2	MSE	MAE	R2
0.002	0.031	0.976	0.003	0.036	0.970	0.000	0.003	0.985

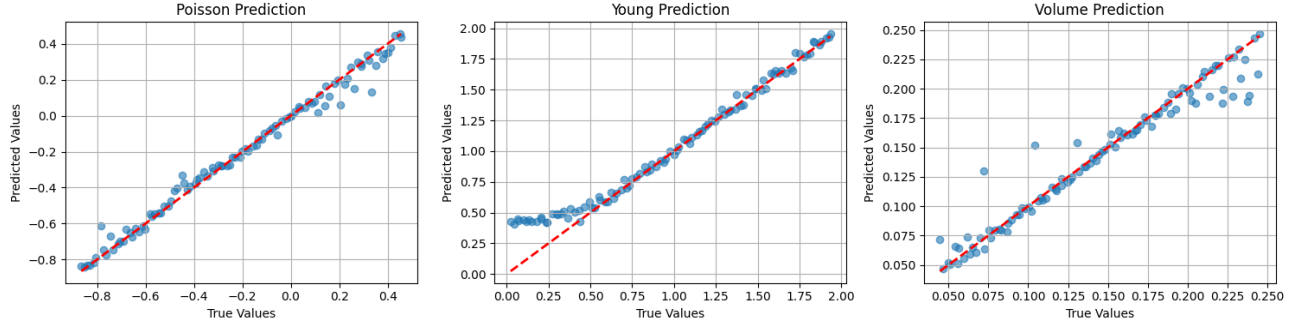


Figure 26: Forward predicted properties from inverse model outputs versus desired properties of the test set, which are true inputs to Inverse A

4.4.3. Inverse Design System B

Forward B, the forward model of System B, features the same architecture as Forward A. However, the training setup configurations were changed to see if performance benefits from a different loss criterion. For this second training loop, the loss criterion is L1Loss (MAE loss). This loss function reflects the error in the original physical units, which is more intuitive for interpretation. The change in loss function caused the model to need additional epochs to achieve convergence. As a result, the number of epochs was set to 250.

The learning curves during the second iteration of forward model training are illustrated in **Figure 27** and the test results can be found in **Table 5**. Even though the model architecture is the same as the first iteration, the learning curves in this iteration are noisier, potentially suggesting that the learning rate is still too high. However, Smith et al. show that that noise which is inherent in small batch stochastic gradient descent in the early training stage can improve generalization for a model [38]. However, the authors also argue that decreasing the learning rate later allows that model to benefit from early noisy exploration while transitioning to a more stable behavior for higher test accuracy. We adopted a similar strategy by setting a high initial learning rate of $5e^{-2}$ and applying a reduce-on-plateau scheduler to gradually lower the learning rate as the training progresses.

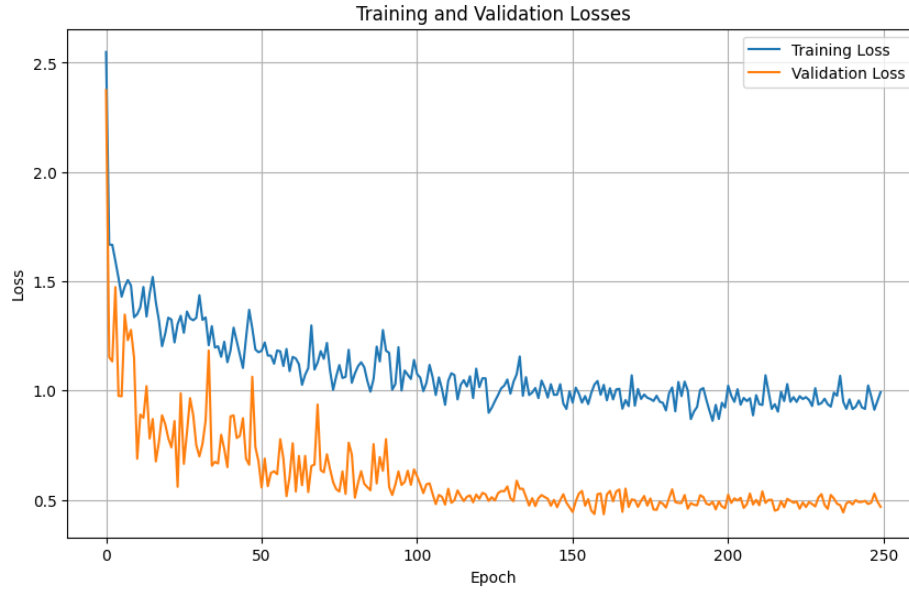


Figure 27: Training and validation loss during second iteration of forward model training

Table 5: Performance metrics on the test set of Forward B

Effective Poisson's ratio			Effective Young's modulus			Volume fraction		
MSE	MAE	R2	MSE	MAE	R2	MSE	MAE	R2
0.001	0.025	0.982	0.004	0.031	0.970	0.000	0.003	0.983

The key benefit of the Forward B is the improvement in MAE metrics across all output properties while maintaining similar values in MSE and R2. In the context of our study, we care more about MAE among the three metrics MSE, MAE and R2 due to its ability to reflect error in physical units.

The inverse model of System B (**Inverse B**) deviates quite significantly from Inverse A, both in terms of architecture and training scheme. Firstly, the architecture of Inverse B has greater depth and higher hidden layer dimensions. The architecture of the inverse model of, which is illustrated in **Figure 28** consists of:

- Input layer: 3 neurons (predicted properties: Poisson's ratio, Young's modulus, volume).
- Hidden layer: 256 neurons linear transformation layer with batch normalization and RELU activation function.
- Hidden layer: 512 neurons linear transformation layer with batch normalization and RELU activation function.
- Hidden layer: 512 neurons linear transformation layer with batch normalization and RELU activation function.

- Hidden layer: 256 neurons linear transformation layer with batch normalization and RELU activation function.
- Output layer: 8 neurons (design parameters: $w_1, w_2, w_3, w_4, h_4, d_1, d_2, d_3$).

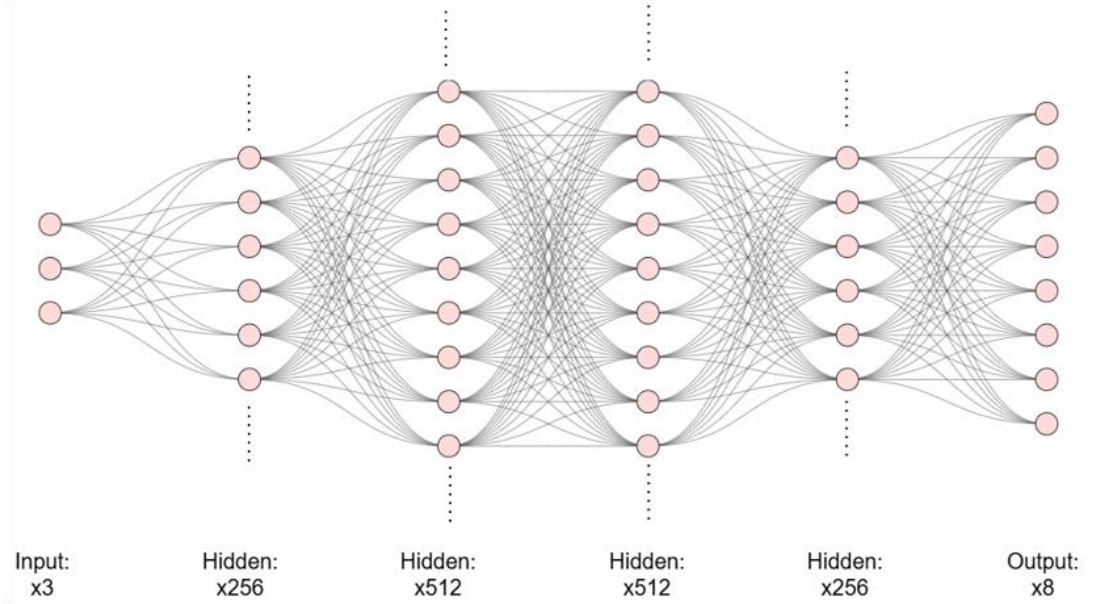


Figure 28: New Inverse FFNN Architecture

The new inverse architecture now has a symmetric structure with higher dimensions for hidden layers. We expect that the introduction of additional architectural complexity will improve the model's performance. Moreover, we removed the Dropout layers. This design choice was made based on the hypothesis that dropout may hinder convergence of training algorithm on shallow networks, which can be observed in the experimental study by Piotrowski et al. [39]. Instead, regularization of the model was implemented with Adam optimizer with decoupled weight decay (also typically known as AdamW) [40]. Weight decay offers more stable and low-variance regularization in comparison with dropout. We observed a significant improvement in performance of the inverse model on the test set with the new trained inverse model, as seen in **Table 7** in the Results section. Finally, the loss function was also changed. Similar to Forward B, the loss criterion used in Inverse B is L1Loss and not MSE loss like Inverse A. In addition, we defined a new penalty term. The penalty term of Inverse A loss function is based on difference between predicted and dataset design parameters that have the same amount of influence on the outcomes, thus too strictly enforcing the prior design ranges. Therefore, we decided to train the inverse model incorporated with a different loss function below:

$$\text{loss} = \beta_1 \cdot L1(\text{Poisson ratio}) + \beta_2 \cdot L1(\text{Young's modulus}) + \beta_3 \cdot L1(\text{volume fraction}) + \beta_4 \cdot \text{penalty},$$

where each $L1(\text{property})$ term measures the difference between the desired property and the forward-predicted property from the inverse-generated design parameters. In addition, the penalty term in this updated loss function is

$L1(\text{inverse predicted design parameters, clamped inverse predicted design parameters})$.

We use the $\beta_1 = \beta_2 = \beta_3 = 1$ and $\beta_4 = 1.23$. These weights allow more control over the impact of each property or penalty. In this case, we place more importance on the MAE (L1Loss) between the raw and clamped design outputs than property losses. To compute the penalty, we first scaled the expected design parameter ranges using the scaler of the inverse model targets. Consequently, we clamp any value in the inverse output tensor that is not within the specified bounds using `torch.clamp`. Finally, we compute the L1 loss between raw and clamped outputs. This encourages the model to generate more parameters within range without forcing them toward the original dataset parameters. The performance metrics on the test set of Inverse B is shown in **Table 6** and **Figures 29** and **30**.

Table 6: Performance metrics on the test set of Inverse B

Effective Poisson's ratio			Effective Young's modulus			Volume fraction		
MSE	MAE	R2	MSE	MAE	R2	MSE	MAE	R2
0.001	0.026	0.991	0.001	0.033	0.992	0.000	0.003	0.991

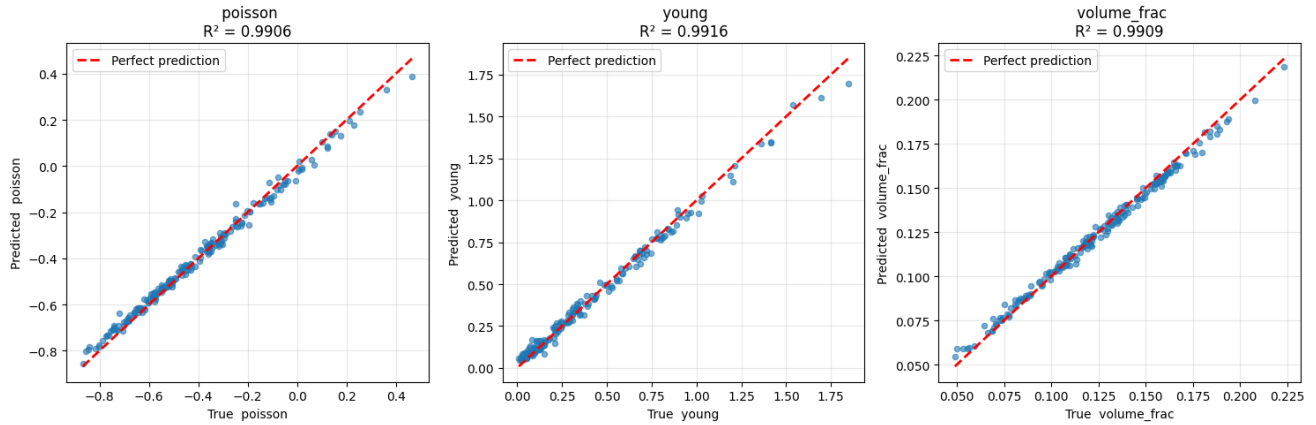


Figure 29: Forward predicted properties from inverse model outputs versus desired properties of the test set, which are true inputs to Inverse B

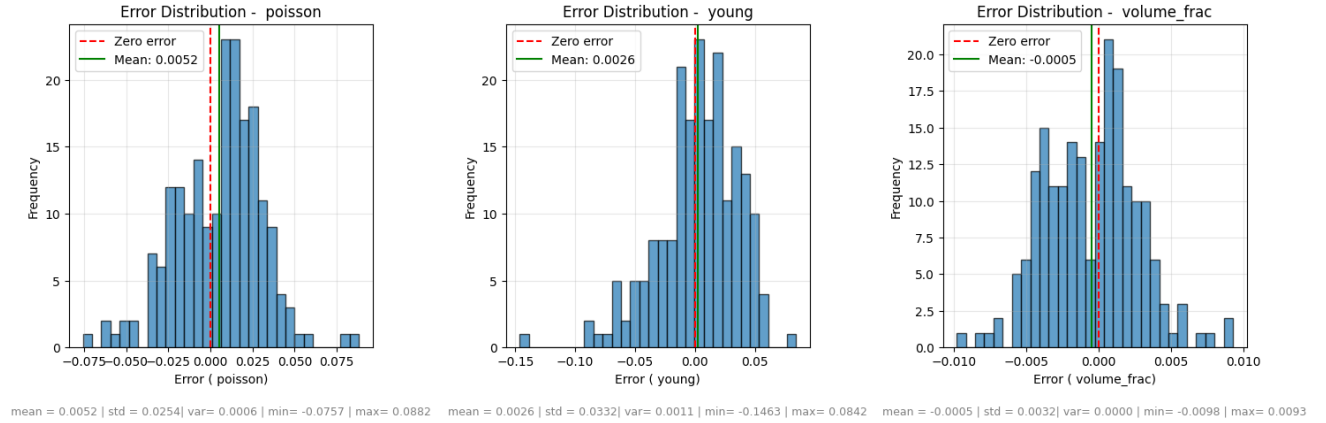


Figure 30: Error distributions of forward predicted properties from inverse model outputs for Inverse B

However, the performance metrics based on the forward predictor only partially reflect the performance of the overall inverse design system. In other words, these metrics alone are not a direct indicator of whether System B has a clear advantage over System A or vice versa. To reliably compare their performances, both System A and System B need to be evaluated with a common testing set, with the properties of the generated designs simulated with FEA and compared against the target properties. The metrics from this process would reflect the true performance of two systems.

4.4.4. Validation with FEA

In the previous section, we have described evaluation of the inverse model with respect to the forward model. However, this evaluation only provides information regarding how aligned the predictions of the inverse model are with the forward model, since the inverse model is trained through forward model predictions. Furthermore, from the inverse model test results, it is clear that the model does not make perfect predictions and thus suffers from slight bias. Therefore, in order to obtain true prediction performance, we need to perform simulations on the predicted designs and compare them with the input properties/forward model predicted properties.

In order to perform this evaluation, we decided to generate 100 new samples of design properties which would be fed into the inverse model. As with the design dataset, we create the samples through LHS. The range of each parameter is defined by the minimum and maximum values observed in the Abaqus-computed properties in the 1000 sample dataset. The used ranges are displayed in **Table 6**, and the resulting design properties distribution can be seen in **Figure 31**.

Table 6: Design properties ranges used for FEM validation

Design property	Minimum value	Maximum Value
Effective Poisson's ratio	-0.874	0.465
Effective Young's modulus	0.009	1.952
Volume fraction	0.044	0.246

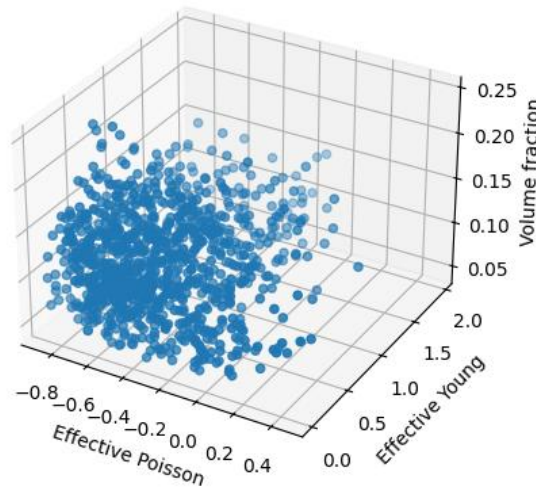


Figure 31: Design properties obtained through LHS

After the new dataset is found and the target properties are passed into the inverse model to obtain predicted design parameters, each mesh is computed and exported into a .inp file. The true design properties are then calculated in Abaqus. Lastly, the true properties are compared to the requested/predicted properties for both inverse models. The distribution of the predicted vs real properties can be seen in **Figure 32** for the old model and in **Figure 33** for the new model.

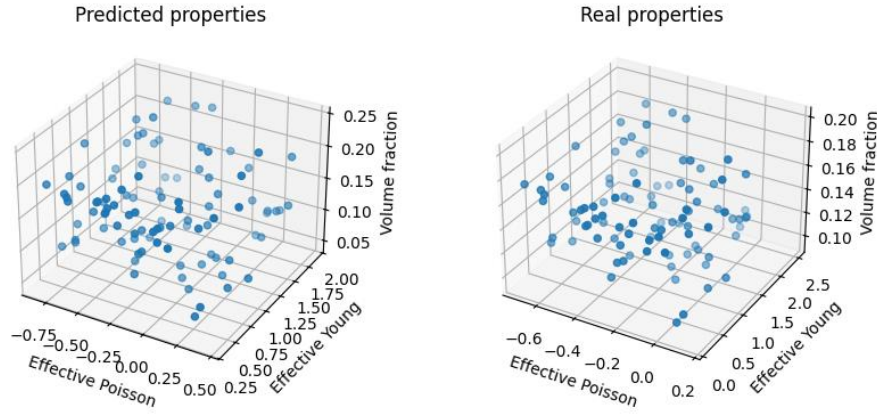


Figure 32: Distribution of predicted and real properties for old inverse model

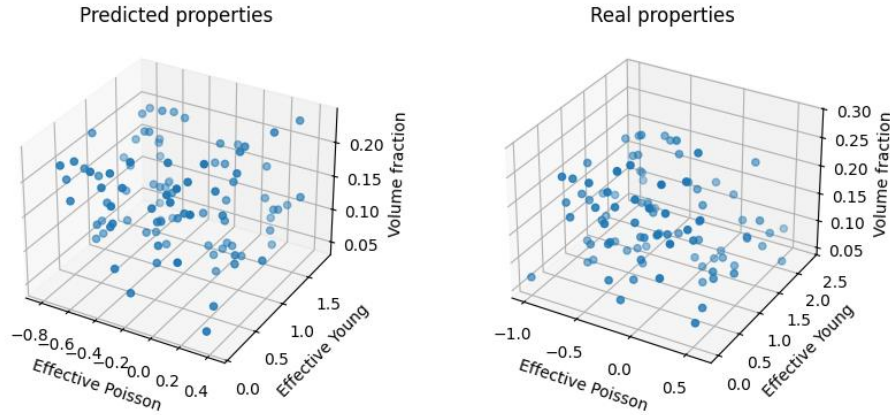


Figure 33: Distribution of predicted and real properties for new inverse model

Lastly, through FEA validation we also confirm that both inverse and forward models reliably generate proper meshes and predict their properties. While both models have been trained on data restricted to parameter ranges which always create proper meshes, the model outputs are not constrained, meaning the predicted designs can be outside of the predefined ranges. On one hand, this could lead to more expressive models, however, on the other, there is no guarantee that the meshes are well defined, i.e., deformed petals or design outside of the RVE bounding box. The results of the comparison and parameter analysis can be found in the **Results** section.

5. Results

5.1. Properties predicted by the forward models

Figures 34, 35, and 36 demonstrate the errors between the forward predicted properties and FEA simulated properties of the generated designs by System A and System B across three metrics including MSE, MAE and R2. We cannot directly compare these metrics numerically because the forward models were not given the same set of designs for the property prediction task. However, the comparative bar graphs show that both systems were able to generate designs and predict volume fraction values very close to the target. On the other hand, more deviation from the targets is observed for effective Poisson ratio and effective Young modulus. Specifically, for both models, MSE and MAE of effective Young modulus approximately double those of Poisson ratio. This may suggest that volume fraction was the easiest property to match and predict, while Young's modulus was the most challenging.

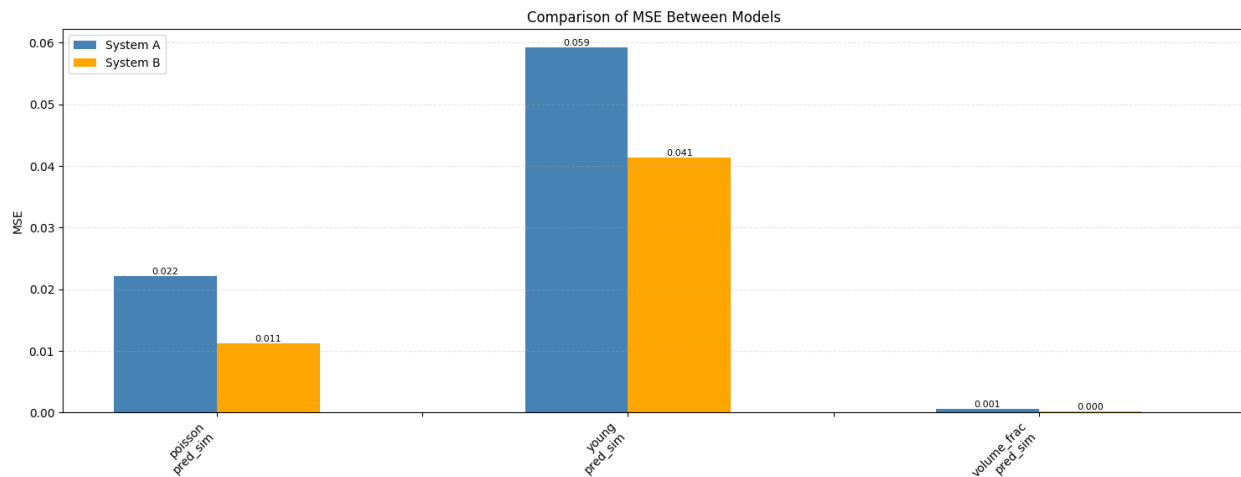


Figure 34: MSE between FEA-simulated properties of the predicted designs and the predicted forward model properties of both systems

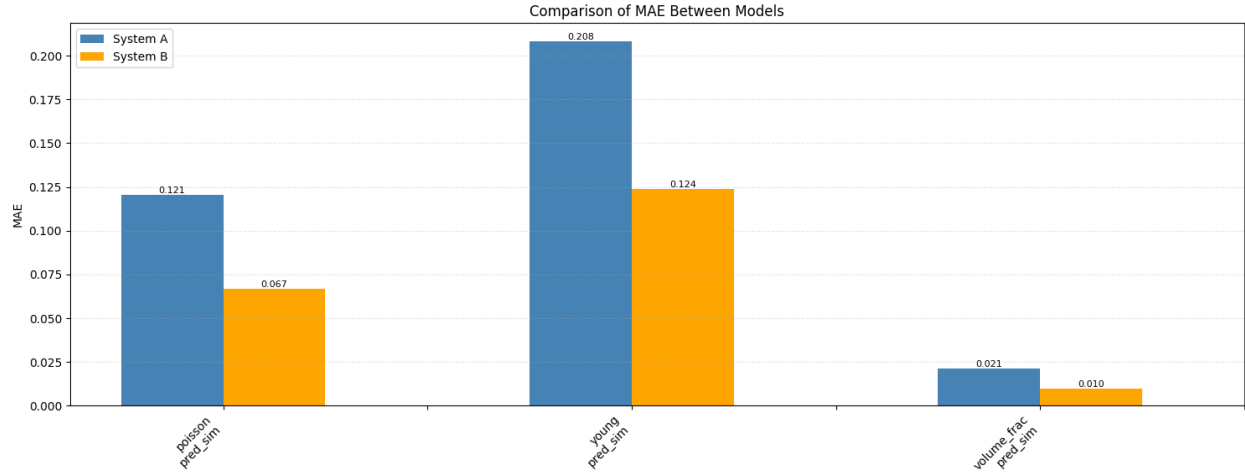


Figure 35: MAE between FEA-simulated properties of the predicted designs and the predicted forward model properties of both systems

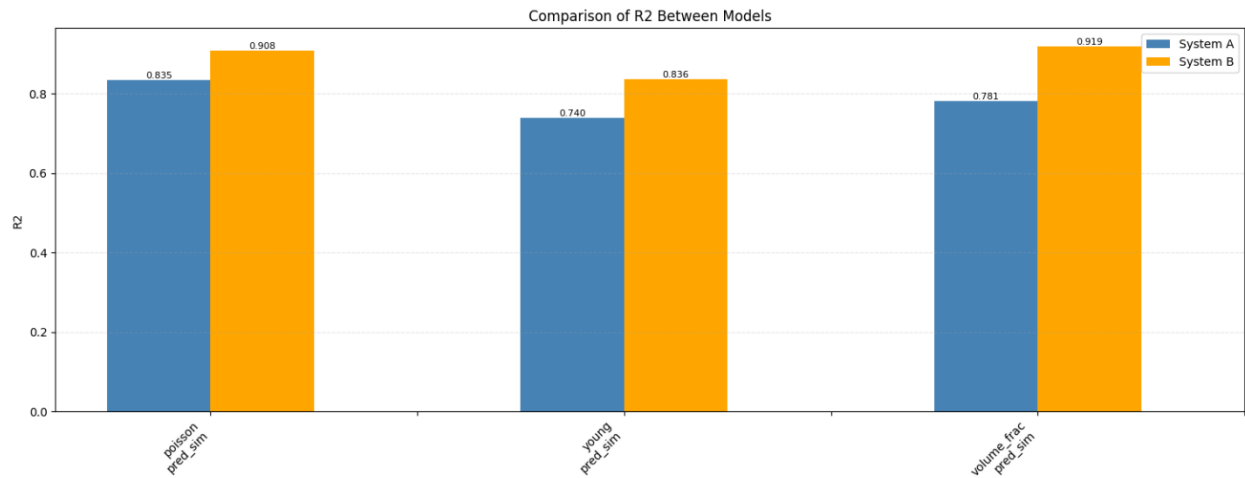


Figure 36: R2 between FEA-simulated properties of the predicted designs and the predicted forward model properties of both systems

5.2. Performance of the inverse design systems

Figures 37, 38, and 39 demonstrate the errors between the forward predicted properties and FEA simulated properties of the generated designs by System A and System B across three metrics including MSE, MAE and R2. The performance metrics of the inverse design systems on the common test set are directly comparable and reliable as they

demonstrate directly the difference between the target properties and the Abaqus-simulated properties of the generated designs. We trust the Abaqus generated results more than the values predicted by the forward model because forward models are only approximators of such complex simulations. The greatest magnitude of error and the smallest magnitude of error are seen in effective Young's Modulus and volume fraction respectively. However, the difference between the MSE and MAE of System A and System B is effective Poisson ratio is the most significant, where the errors produced by System A is approximately double those of System B. However, interestingly, in terms of R2, the values of both models are fairly similar with the exception of volume fraction, where that of System B is 20% higher than System A.

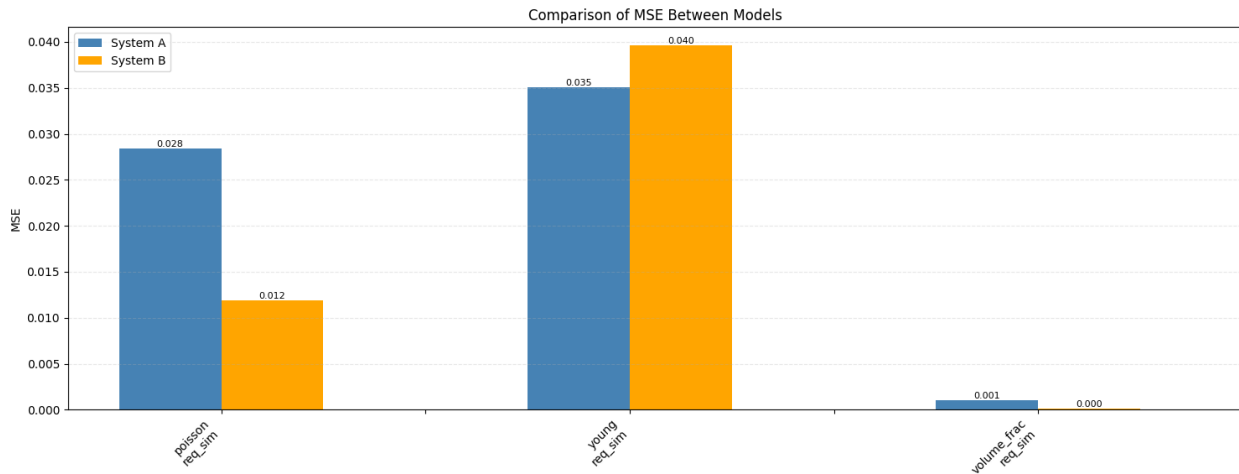


Figure 37: MSE between FEA-simulated properties of the predicted designs and the target properties for both the inverse model of both systems

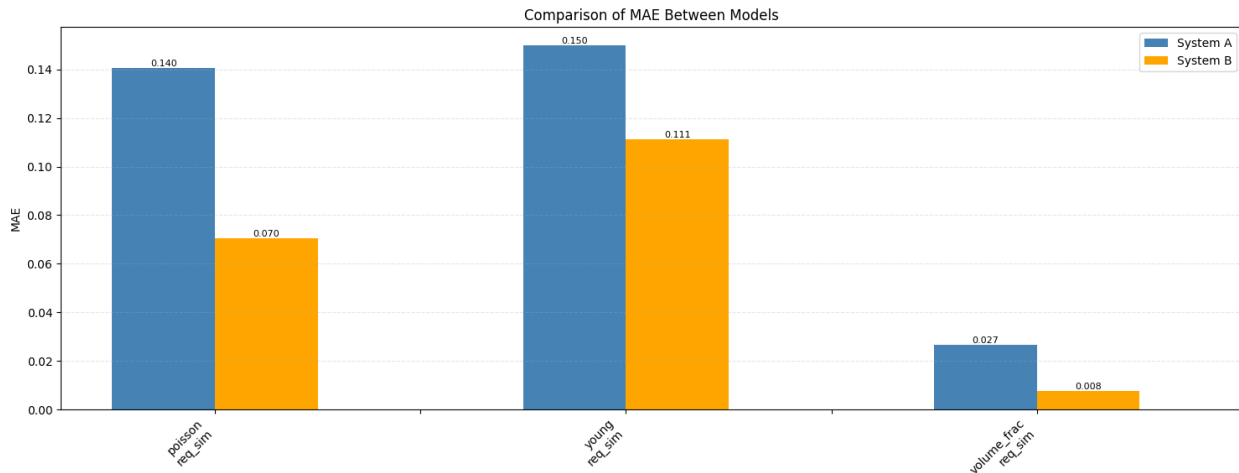


Figure 38: MAE between FEA-simulated properties of the predicted designs and the target properties for the inverse model of both systems

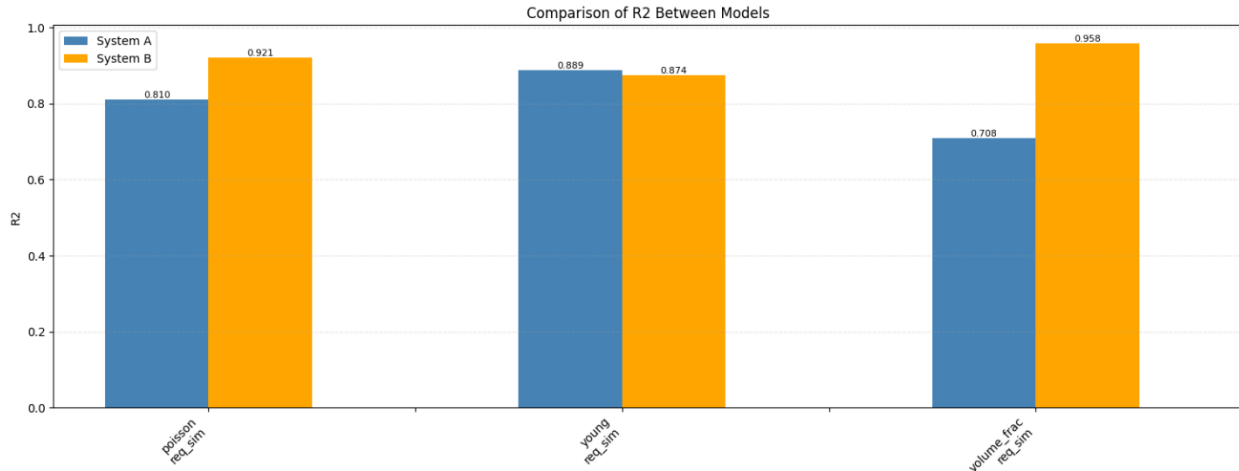


Figure 39: R2 between FEA-simulated properties of the predicted designs and the target properties for the inverse model of both systems

5.3. Comparing the amount of problematic designs

With a set of 100 inputs, we can check the number of possibly problematic design parameters for both the old and new models, using the same parameter ranges defined above. Our initial assumption is for Inverse A to perform better due to its tanh layer. The number of designs with out-of-bound parameters can be seen in **Figure 40**.

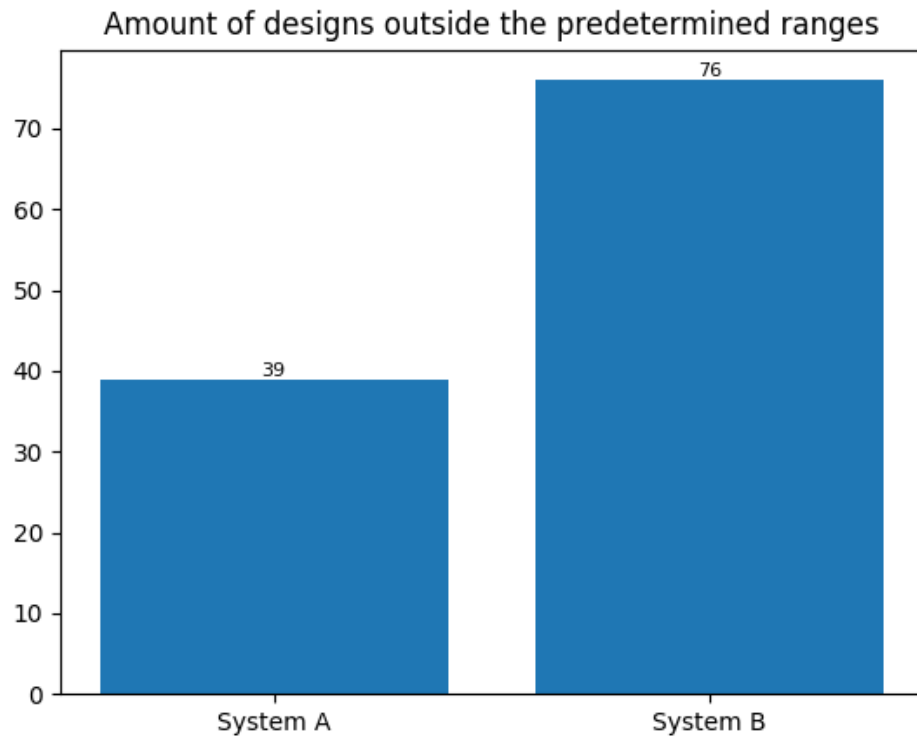


Figure 40: The number of designs with out-of-bound parameters

From the figure it is confirmed that our assumptions were correct, as only 39% of Inverse A designs are out of bounds, while 76% of Inverse B predictions violate the ranges. Furthermore, in order to ensure that the possibly problematic meshes are proper, i.e., they do not extend out of the RVE and the petals are not overlapping, we generated meshes through the design parameters and performed a visual check. Overall, no out-of-bound designs from either model were detected during the visual check. Lastly, we analyze the out-of-bound distributions of each design parameter. The results can be found in **Figure 41**.

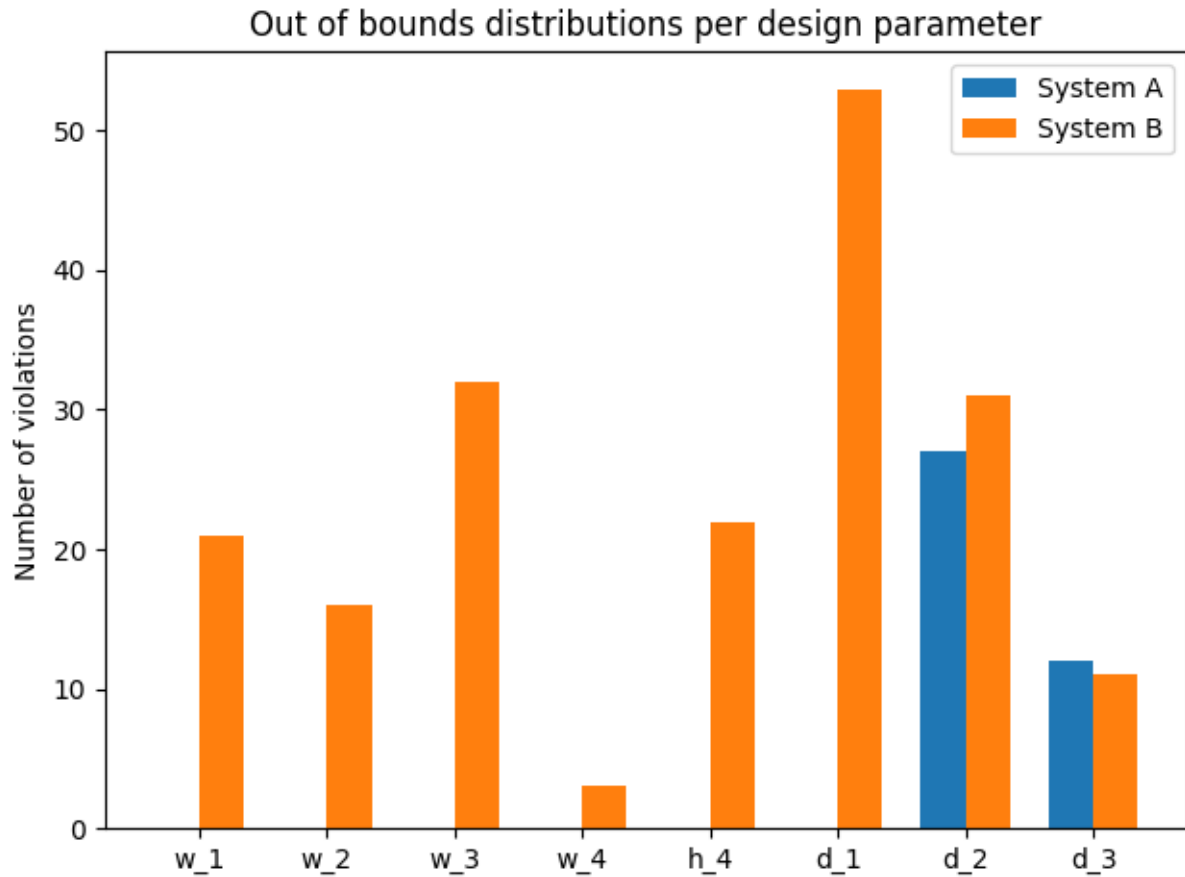


Figure 41: The number of out-of-bound violations for each parameter

From the graph above, we can see that Inverse A only violated parameters d_2 and d_3 , which correspond to petal and connective bar thickness respectively. On the other hand, Inverse B violated every parameter at least once. Furthermore, parameter d_1 was out of range for over half of the generated designs.

6. Discussion

6.1. Performance comparison between System A and B

When comparing forward models, A and B, we can say that they have no significant difference in performance on their respective test sets, with model B having slightly more superior results in terms of Poisson ratio. In terms of R^2 , both score over 97% meaning that the models explain almost all variation in the data. However, more direct comparison using a common test set could have been done to further distinguish the level of performance between the two implementations. On the other hand, the gap between performance is more visible for inverse models given that both models were given a common test set and were compared against FEA results. Inverse B outperforms Inverse A at every metric, except for effective Young's modulus MSE and R^2 . However, Inverse B demonstrates a clearly superior performance in Poisson ratio and volume fraction, especially in MAE, the most important metric for our study across all three properties. For that reason, we consider that the benefits of Inverse B outweigh its drawbacks and Inverse B is more suitable for practical applications than Inverse A.

6.2. Limitations

However, the models still present several limitations, suggesting areas of improvement. Firstly, through the figures presented in results, it is clear that the model could not capture the relationship between effective Young's modulus and the design parameters. Furthermore, it should be noted that effective Young's modulus has the largest observed distribution out of any property, meaning that the property is susceptible to higher variance. Another possible contributor to this phenomenon is the forward models. Based on the error distribution of Forward B in Figure 29, we can see that there are occasional outliers where the absolute error for a prediction can reach close to 0.15 for effective Young's modulus. Error distributions of effective Poisson ratio and volume fractions have significantly smaller ranges, with absolute errors reaching 0.075 and 0.01 in the worst case. This means that the forward models are less stable when predicting effective Young's modulus, possibly leading to the inverse models performing poorly at effective Young's modulus as well. This suggests that the forward model remains too simple and unstable as a properties approximator in comparison to FEA simulations. Another issue we identified is the way we handled dataset generation. We used LHS given the parameter design ranges, which does not inherently guarantee that the distributions for the resulting properties will be similar. This is likely caused the range of effective Young's modulus to be the largest, while the majority of data points are on the lower range of effective Young's modulus. Without sufficient data in the higher range, the model likely did not capture the underlying pattern well.

Secondly, the findings from the parameter analysis suggest that Inverse B violates bound not only for all parameters, but also much more frequently. These violations influence the positions of the control points, petal height, as well as petal and connecting bar thickness. Therefore, with some specific combinations of parameters, it could potentially generate improper designs. On the other hand, Inverse A is much more restricted within our ranges and only the thickness parameters are violated. Furthermore, it is important to note that neither inverse system generated any improper meshes out of the 100 FEA validation samples, indicating that generating invalid designs might be a rare occurrence. Nonetheless, in order to assure that potentially problematic meshes are proper, RVE bounding box inspection and petal intersection should be performed.

The methods that we have chosen to apply here bear some similarities to existing work. Many researchers opted to generate a parametric dataset similar to our methods [24], [27], [28]. The approach is sensible as robust datasets that are suitable for ML are uncommon. Moreover, having implemented our own ML framework, the concept of deploying a forward and an inverse model is valid. By using a forward model as an approximator for numerical calculations, the process of designing new metamaterials would require considerably less computational power. Therefore, the foundational ideas of inverse design are solid. However, there is still room for development in the execution. Having examined the performance of our models, their performance is found to be adequate at best. The error values of our model are non-insignificant and improvements to the models can be made. For example, a more accurate forward model will likely improve the overall performance of the system. This can be achieved with a larger dataset, modifications to the training and testing process, or by changing the model architecture.

7. Extra deliverables

For our extra deliverable we have decided to implement a PyQT GUI application. It can be run by executing the file *main.py* found in the project's root folder. The initial window is shown in **Figure 42**. The application allows the user to pass a CSV file with the target design properties as an input by selecting the option Predict from CSV in the File dropdown menu. The inputs are stored in the *input* folder by default, but other directories can be used as well. The input CSV needs to contain the following columns: poisson, young, volume_frac. After an input is passed, the program outputs design obtained from inverse system B into the folder *outputs* with the name *{name of the input file}_results.csv*. The columns present in the CSV, and their description can be found in **Table 8**.

Table 8: GUI output CSV columns and their description

Column names	Description
w1, w2, w3, w4, h4, d1, d2, d3	Predicted design by the inverse model
poisson_req, young_req, volume_frac_req	User requested design properties
poisson_predicted, young_predicted, volume_frac_predicted	Predicted design properties by the forward model
possibly_problematic	A Boolean value determining whether the predicted design parameters lie outside of our sampling ranges. If true, the mesh is potentially improper.

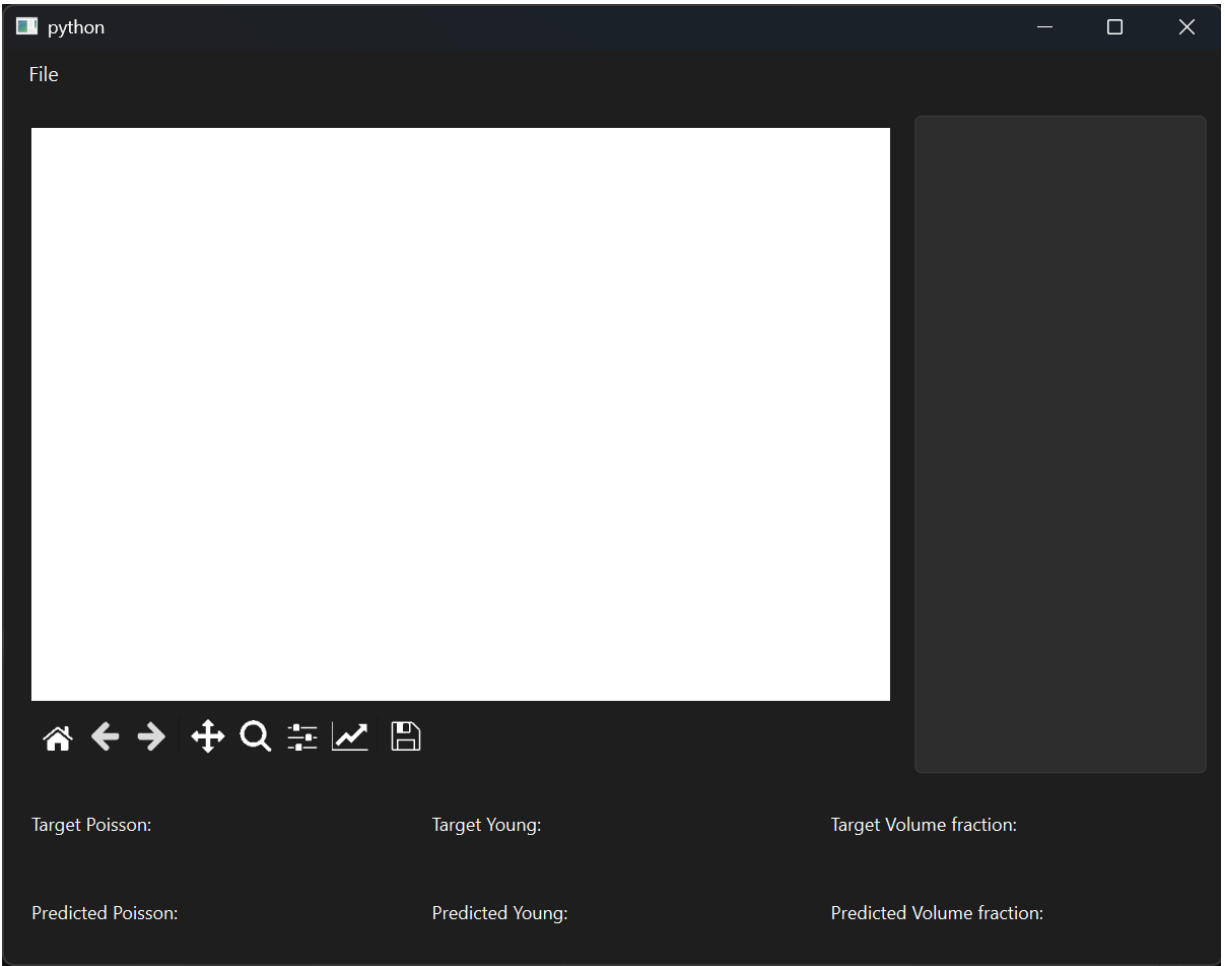


Figure 42: Application GUI when initialized

Once the predictions have been written into the output CSV, the GUI state is updated and gains additional functionality. The new GUI state can be seen in **Figure 43**. In this state, each predicted design can be examined in the GUI. Different designs can be chosen by selecting them from the right list panel. Each design provides both numerical and visual information regarding the design:

- Numerical – provides comparison between each user requested and forward model predicted parameter in the bottom panel
- Visualization – draws the interior + exterior design curves and the RVE bounding box of the predicted design. The design is drawn on a matplotlib plot, meaning it is interactable, i.e., the user can pan and zoom into the design.

From the design selection list, some portions of designs will be highlighted with orange. These designs are marked as potentially problematic, as the design parameters are outside of the predefined ranges our parameters were sampled from. Therefore, these

designs cannot be guaranteed to be proper, for instance, they can extend past the RVE bounding box and therefore should be visually examined.

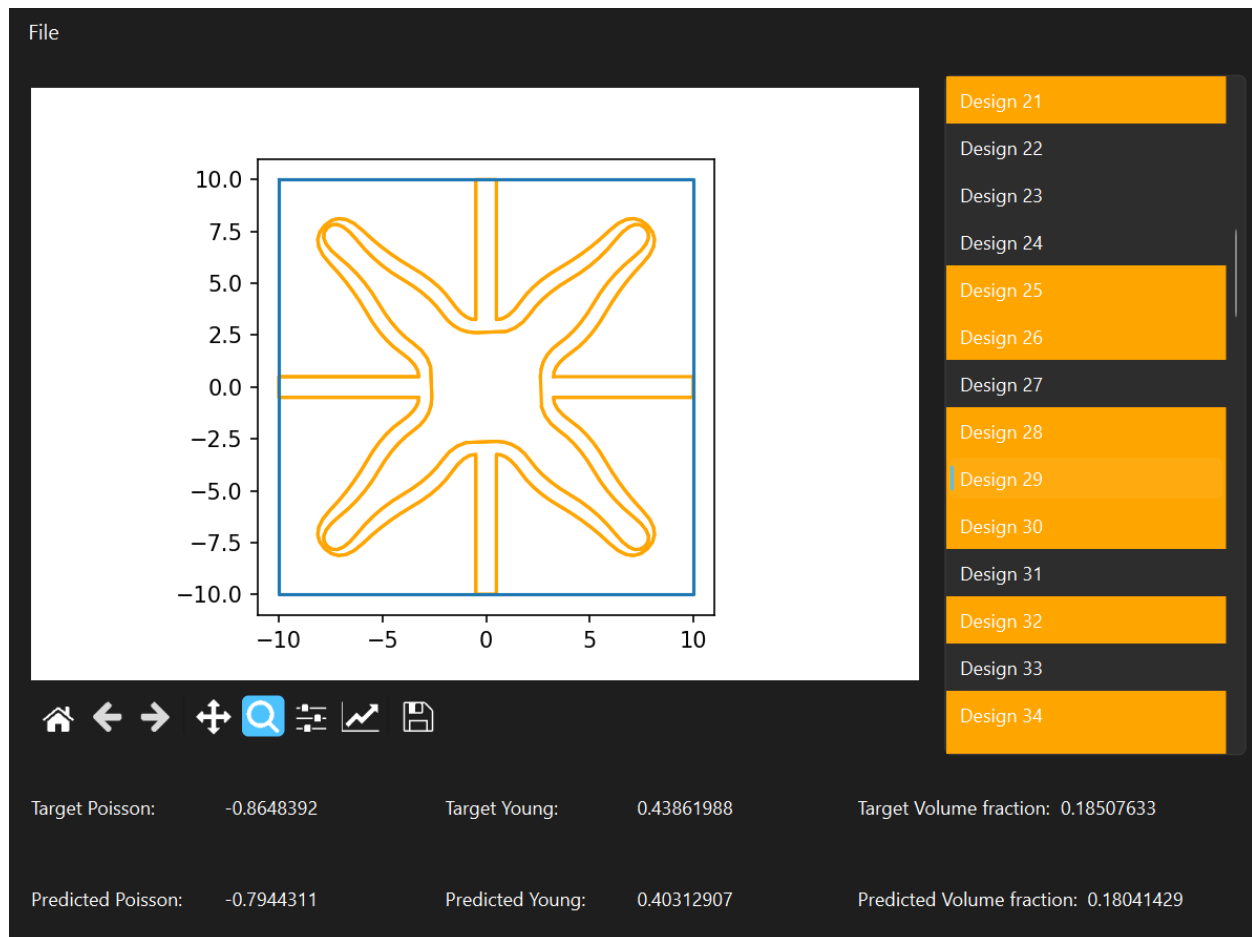


Figure 43: GUI state after design prediction

8. Conclusions and Outlooks

In this report, we have described the implementation of two inverse machine learning systems capable of predicting metamaterial designs based on desired design properties, i.e., Poisson's ratio, Young's modulus and volume fraction. Furthermore, we described a PyQT GUI application which allows the user to create design predictions through our inverse model, as well as visual inspection of the generated designs.

Our pipeline begins with design generation, for which we opted for tetra-petals implemented through non-uniform rational b-splines. These designs often exhibit auxeticity, as well as isotropy due to their symmetry, which are both desirable metamaterial traits. Furthermore, this design can produce varied and flexible designs, as it is described through eight separate parameters. Afterwards, the metamaterial properties were obtained through Abaqus uniaxial tensile simulations which we used for training and validation of both inverse systems.

All the ML models have been implemented through PyTorch as deep neural networks. In the framework, the inverse model uses the target properties as inputs and outputs the predicted tetra-petal design parameters. The forward model is utilized alongside the inverse model, as it predicts the properties of the inverse designs which provides an approximation of the actual properties to the user without running any FEA simulations.

In the report, descriptions of two inverse systems, inverse system A and inverse system B, were provided. Inverse system A serves as our initial implementation of the prediction pipeline which was designed based on existing literature. Afterwards, the inverse system B was developed which tried to fix the shortcomings of the first system.

The inverse models of both systems were compared through FEA validation which highlighted the true model deviations between the properties of the predicted designs and the requested properties. From the comparison, we have discovered that inverse model B shows significant improvement in most metrics across all parameters, apart from Young's modulus which has a slight decrease in the R^2 and MSE metrics. These findings indicate that inverse A has better performance, however, improvements could still be made. Compared to other parameters, Young's modulus metrics are worse (MSE: 0.04, MAE: 0.111, R^2 : 0.874) and compared to volume fraction, the MAE is increased by 1288%. Furthermore, the FEA highlighted that both forward models have significantly increased MSE and MAE for Poisson's ratio and Young's modulus which can further amplify the mispredictions of the inverse models. Lastly, inverse system A is potentially more susceptible to generating improper meshes, as the predicted design parameters often exceed our parameters' bounds (of which proper meshes are not guaranteed).

Therefore, future works could expand on our study by refining the inverse system further. We suspect that improving the forward model could positively affect the performance of the inverse model.

From the discussion of our project results in **Section 6.1** and **Section 6.2**, we have identified three possible strategies to address the limitations of the inverse system design. Since we suspect that the instability of the forward model is a factor behind the worse performance of the model, experimentation with a more complex forward model or modified training configurations is needed. Better regularization is also believed to help with lessening outliers during predictions, improving stability of predictions. Regarding the problem of high concentration of data around a specific part of a range, we can apply data augmentation to make sure we have more uniform concentration of data on all axes. Finally, to improve the overall robustness of the model and prevent faulty design, adversarial training strategies can be implemented.

Lastly, the mesh generation itself could be expanded upon. While tetra-petals offer a wide range of designs, the variety could be further increased by considering different parametric curves. A natural extension would be the implementation of n-petals, for instance, tri-petals or hexa-petals.

9. Personal evaluation

Throughout the course of this project, all of us have gained new skills or experience. We learned how to interpret information from papers, in order to implement our own algorithms, for instance the generation of the tetra-petal, or the ML models. Additionally, we have become more skilled in working and reviewing Python code, as well as creating a working pipeline for our mesh generation and model training/testing. Regarding the paper, we have all gained a better understanding of how to write a report of this scale in general and how to divide the work between ourselves properly.

Working with the tetra-petal taught us more about the behavior of NURBS and how applying 4-fold symmetry works. Additionally, since we were creating the design generator from scratch, we also needed to find a way to export it into Abaqus and automatically run simulations on all our designs in bulk.

Working on ML models was insightful as well, since it was the first time interacting with them for some of us. We have gained a better understanding of the inner workings of deep learning models which we have implemented through PyTorch. Furthermore, we analyzed these models through fine-tuning, using different layers in the architecture, and employing different losses during model training. Lastly, we also learned about regression metrics and how to meaningfully compare the performance between our two inverse models.

Overall, we would say that we are very content with our performance on this project. We found the topic both interesting and challenging, however, we believe that 60 pages for this project was too much. While we have hit our goal, we think that the amount of information which we had to write needlessly complicates the structure and makes the contents less understandable due to its length. Additionally, the 60 minimum page limit goes against the standard practice of scientific papers which try to present information as concisely as possible, due to maximum page limit. Furthermore, while the COE presentations were interesting, we suggest that both workshops should be split into two sessions, such that each student would only need to attend half the presentations. Having to stay in a room for 4 hours with only an 8-minute break in the middle was not ideal. Furthermore, splitting the workshop into multiple sessions would allow students to choose which time/date suits them better, since there were 11 people that would have preferred a time slot on Friday instead of Wednesday.

Lastly, we would want to thank our project advisor, Rauf Talha, for guiding us through the project and providing us with advice and materials.

10. References

- [1] A. Challapalli, D. Patel, and G. Li, "Inverse machine learning framework for optimizing lightweight metamaterials," *Mater. Des.*, vol. 208, p. 109937, Oct. 2021, doi: 10.1016/j.matdes.2021.109937.
- [2] C. Zhang and Y. F. Zhao, "A critical review on the application of machine learning in supporting auxetic metamaterial design," *J. Phys. Mater.*, vol. 7, no. 2, p. 022004, Apr. 2024, doi: 10.1088/2515-7639/ad33a4.
- [3] Z.-P. Wang, L. H. Poh, Y. Zhu, J. Dirrenberger, and S. Forest, "Systematic design of tetra-petals auxetic structures with stiffness constraint," *Mater. Des.*, vol. 170, p. 107669, May 2019, doi: 10.1016/j.matdes.2019.107669.
- [4] R. Kumar, M. Kumar, J. S. Chohan, and S. Kumar, "Overview on metamaterial: History, types and applications," *Mater. Today Proc.*, vol. 56, pp. 3016–3024, 2022, doi: 10.1016/j.matpr.2021.11.423.
- [5] H. R. Hosseinzadeh, "Metamaterials in Medicine: A New Era for Future Orthopedics," *Orthop. Res. Online J.*, vol. 2, no. 5, May 2018, doi: 10.31031/OPROJ.2018.02.000549.
- [6] G. Palma, H. Mao, L. Burghignoli, P. Göransson, and U. Iemma, "Acoustic Metamaterials in Aeronautics," *Appl. Sci.*, vol. 8, no. 6, p. 971, June 2018, doi: 10.3390/app8060971.
- [7] A. Ahmad, A. Asad, P. Pacpaco, C. Thampongphan, and M. Hasibul Hasan, "Application of Metamaterial in Renewable Energy: A Review," *Int. J. Eng. Mater. Manuf.*, vol. 9, no. 2, pp. 60–80, Apr. 2024, doi: 10.26776/ijemm.09.02.20243.04.
- [8] R. Kumar, M. Kumar, J. S. Chohan, and S. Kumar, "Overview on metamaterial: History, types and applications," *Mater. Today Proc.*, vol. 56, pp. 3016–3024, 2022, doi: 10.1016/j.matpr.2021.11.423.
- [9] R. Çalışkan, S. S. Gültekin, D. Uzer, and Ö. Dündar, "A Microstrip Patch Antenna Design for Breast Cancer Detection," *Procedia - Soc. Behav. Sci.*, vol. 195, pp. 2905–2911, July 2015, doi: 10.1016/j.sbspro.2015.06.418.
- [10] B. Xu, Y. Li, and Y. Kim, "Classification of Finger Movements Based on Reflection Coefficient Variations of a Body-Worn Electrically Small Antenna," *IEEE Antennas Wirel. Propag. Lett.*, pp. 1–1, 2017, doi: 10.1109/LAWP.2017.2681099.
- [11] R. Liu, C. Ji, Z. Zhao, and T. Zhou, "Metamaterials: Reshape and Rethink," *Engineering*, vol. 1, no. 2, pp. 179–184, June 2015, doi: 10.15302/J-ENG-2015036.
- [12] Z. Zou, F. Xu, Y. Pan, X. Niu, T. Fang, and C. Zeng, "Parametric Analysis and Multi-Objective Optimization of Pentamode Metamaterial," *Sustainability*, vol. 15, no. 4, p. 3421, Feb. 2023, doi: 10.3390/su15043421.
- [13] L. Széles, R. Horváth, and J. P. Rádics, "Design and Study of Fractal-Inspired Metamaterials with Equal Density Made from a Strong and Tough Thermoplastic," *Polymers*, vol. 15, no. 12, p. 2650, June 2023, doi: 10.3390/polym15122650.
- [14] L. Széles, R. Horváth, J. P. Rádics, L. Széles, R. Horváth, and J. P. Rádics, "Design and Study of Fractal-Inspired Metamaterials with Equal Density Made from a Strong and Tough Thermoplastic," *Polymers*, vol. 15, no. 12, June 2023, doi: 10.3390/polym15122650.
- [15] C. Liu and B. Wang, "Shellular Metamaterial Design via Compact Electric Potential Parametrization," 2025, *arXiv*. doi: 10.48550/ARXIV.2511.04025.

- [16] D. Yago, G. Sal-Anglada, D. Roca, J. Cante, and J. Oliver, "Machine learning in solid mechanics: Application to acoustic metamaterial design," *Int. J. Numer. Methods Eng.*, vol. 125, no. 14, p. e7476, July 2024, doi: 10.1002/nme.7476.
- [17] Z.-P. Wang and L. H. Poh, "Optimal form and size characterization of planar isotropic petal-shaped auxetics with tunable effective properties using IGA," *Compos. Struct.*, vol. 201, pp. 486–502, Oct. 2018, doi: 10.1016/j.compstruct.2018.06.042.
- [18] A. Zunger, "Inverse design in search of materials with target functionalities," *Nat. Rev. Chem.*, vol. 2, no. 4, p. 0121, Mar. 2018, doi: 10.1038/s41570-018-0121.
- [19] X. Zheng, X. Zhang, T. Chen, and I. Watanabe, "Deep Learning in Mechanical Metamaterials: From Prediction and Generation to Inverse Design," *Adv. Mater.*, vol. 35, no. 45, p. 2302530, Nov. 2023, doi: 10.1002/adma.202302530.
- [20] C. S. Ha *et al.*, "Rapid inverse design of metamaterials based on prescribed mechanical behavior through machine learning," *Nat. Commun.*, vol. 14, no. 1, p. 5765, Sept. 2023, doi: 10.1038/s41467-023-40854-1.
- [21] Y. Wang, Q. Zeng, J. Wang, Y. Li, and D. Fang, "Inverse design of shell-based mechanical metamaterial with customized loading curves based on machine learning and genetic algorithm," *Comput. Methods Appl. Mech. Eng.*, vol. 401, p. 115571, Nov. 2022, doi: 10.1016/j.cma.2022.115571.
- [22] W. Ma, F. Cheng, and Y. Liu, "Deep-Learning-Enabled On-Demand Design of Chiral Metamaterials," *ACS Nano*, vol. 12, no. 6, pp. 6326–6334, June 2018, doi: 10.1021/acsnano.8b03569.
- [23] M. Abu-Mualla and J. Huang, "Inverse design of 3D cellular materials with physics-guided machine learning," *Mater. Des.*, vol. 232, p. 112103, Aug. 2023, doi: 10.1016/j.matdes.2023.112103.
- [24] E. Ashalley *et al.*, "Multitask deep-learning-based design of chiral plasmonic metamaterials," *Photonics Res.*, vol. 8, no. 7, pp. 1213–1225, 2020, doi: 10.1364/PRJ.388253.
- [25] H. Pahlavani, M. Amani, M. C. Saldívar, J. Zhou, M. J. Mirzaali, and A. A. Zadpoor, "Deep learning for the rare-event rational design of 3D printed multi-material mechanical metamaterials," *Commun. Mater.*, vol. 3, no. 1, p. 46, July 2022, doi: 10.1038/s43246-022-00270-2.
- [26] M. A. Bessa, P. Glowacki, and M. Houlder, "Bayesian Machine Learning in Metamaterial Design: Fragile Becomes Supercompressible," *Adv. Mater.*, vol. 31, no. 48, p. 1904845, Nov. 2019, doi: 10.1002/adma.201904845.
- [27] J. K. Wilt, C. Yang, and G. X. Gu, "Accelerating Auxetic Metamaterial Design with Deep Learning," *Adv. Eng. Mater.*, vol. 22, no. 5, p. 1901266, May 2020, doi: 10.1002/adem.201901266.
- [28] A. P. Garland, B. C. White, S. C. Jensen, and B. L. Boyce, "Pragmatic generative optimization of novel structural lattice metamaterials with machine learning," *Mater. Des.*, vol. 203, p. 109632, 2021, doi: <https://doi.org/10.1016/j.matdes.2021.109632>.
- [29] V. Pareto, *Manual of political economy: a critical and variorum edition*. OUP Oxford, 2014.
- [30] A. Álvarez-Trejo, E. Cuan-Urquizo, D. Bhate, and A. Roman-Flores, "Mechanical metamaterials with topologies based on curved elements: An overview of design, additive manufacturing and mechanical properties," *Mater. Des.*, vol. 233, p. 112190, Sept. 2023, doi: 10.1016/j.matdes.2023.112190.

- [31] S. Khakalo, V. Balobanov, and J. Niiranen, "Modelling size-dependent bending, buckling and vibrations of 2D triangular lattices by strain gradient elasticity models: Applications to sandwich beams and auxetics," *Int. J. Eng. Sci.*, vol. 127, pp. 33–52, June 2018, doi: 10.1016/j.ijengsci.2018.02.004.
- [32] J. Zhang, W. Geng, and M. Yao, "Chiral petal honeycomb metamaterial structures: Biomimetic design and application in vascular stents," *Biomater. Adv.*, vol. 178, p. 214453, Jan. 2026, doi: 10.1016/j.bioadv.2025.214453.
- [33] S. Kumar, S. Tan, L. Zheng, and D. M. Kochmann, "Inverse-designed spinodoid metamaterials," *Npj Comput. Mater.*, vol. 6, no. 1, p. 73, June 2020, doi: 10.1038/s41524-020-0341-6.
- [34] G. Moon, J. Choi, C. Lee, Y. Oh, K. H. Kim, and D. Kim, "Machine learning-based design of meta-plasmonic biosensors with negative index metamaterials," *Biosens. Bioelectron.*, vol. 164, p. 112335, 2020.
- [35] NVIDIA, "Get Started With Deep Learning Performance." Feb. 2023. [Online]. Available: <https://docs.nvidia.com/deeplearning/performance/pdf/Getting-Started-With-Deep-Learning-Performance.pdf>
- [36] H. Anysz, A. Zbiciak, and N. Ibadov, "The Influence of Input Data Standardization Method on Prediction Accuracy of Artificial Neural Networks," *Procedia Eng.*, vol. 153, pp. 66–70, 2016, doi: 10.1016/j.proeng.2016.08.081.
- [37] M. Z. Al-Faiz, A. A. Ibrahim, and S. M. Hadi, "The effect of Z-Score standardization (normalization) on binary input due the speed of learning in back-propagation neural network," *Iraqi J. Inf. Commun. Technol.*, vol. 1, no. 3, pp. 42–48, Feb. 2019, doi: 10.31987/ijict.1.3.41.
- [38] S. Smith, E. Elsen, and S. De, "On the Generalization Benefit of Noise in Stochastic Gradient Descent," in *Proceedings of the 37th International Conference on Machine Learning, PMLR 119:9058-9067*, in Proceedings of Machine Learning Research. PMLR, July 2020, pp. 9058–9067. [Online]. Available: <https://proceedings.mlr.press/v119/smith20a.html>
- [39] A. P. Piotrowski, J. J. Napiorkowski, and A. E. Piotrowska, "Impact of deep learning-based dropout on shallow neural networks applied to stream temperature modelling," *Earth-Sci. Rev.*, vol. 201, p. 103076, Feb. 2020, doi: 10.1016/j.earscirev.2019.103076.
- [40] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," Jan. 04, 2019, *arXiv*: arXiv:1711.05101. doi: 10.48550/arXiv.1711.05101.